



VELS





INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR, ADYAR - CHENNAI

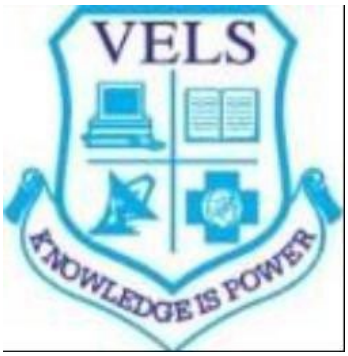
Adaptive AI-Powered Placement Preparation Platform Using Multi-Agent Intelligence for Personalized Career Readiness

FINAL YEAR PROJECT
PHASE-I REPORT
Submitted by

Sathya Sri R (23619243)

in Partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY CSE
IN ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF ENGINEERING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project "Adaptive AI-Powered Placement Preparation Platform Using Multi-Agent Intelligence for Personalized Career Readiness" is the bonafide work of Sathya Sri R (23619243) who carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD
Dr. K. Ulagapriya M.E., Ph.D
Associate Professor & Head
Department of AI & ML
School of Engineering
VISTAS
Chennai-600 117

SIGNATURE OF THE SUPERVISOR
Dr. G. Sangeetha M.E., Ph.D
Assistant Professor
Department of AI & ML
School of Engineering
VISTAS
Chennai-600 117

Submitted for the Project Work and Viva - Voce examination held on …… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

It is a great delight and a privilege to acknowledge the help and support we received from the people around us in making this project a success. The infrastructural support and the laboratory facilities provided were a constant motivation in completing this work. For this we thank our **FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH**, with great pleasure.We are grateful to **our HEAD OF THE DEPARTMENT, Dr. K. Ulagapriya,M.E.,Ph.D**, for continuous support throughout the project, from advice in the early stages of conceptual inception to encouragement through the later stages. We extend our warmest thanks to our **INTERNAL GUIDE, Dr.G. Sangeetha ,M.E.,Ph.D**, who guided us patiently and gave us valuable advice and help. We also thank our **PROJECT COORDINATOR Ms.S.M.Sowmiya, M.E.,(Ph.D)**, for organising the review process. We thank the staff members, the Librarian and the non-teaching staff of the Computer Centre, who supported us throughout. Last, we thank our parents and friends for their support and interest, which encouraged us to go our own way. Without them we would not have been able to complete this project.

SATHYA SRI R (23619243)

TABLE OF CONTENTS

SL.NO	TITLE	PAGE NO
	ABSTRACT	**7**
	LIST OF FIGURES	**8**
	LIST OF TABLES	**9**
	LIST OF ABBREVIATIONS	**11**
01	**INTRODUCTION** 1.1 GENERAL 1.2 PROBLEM STATEMENT 1.3 OBJECTIVES 1.4 SCOPE AND MODULES 1.5 ORGANISATION OF THE REPORT 1.6 SUMMARY	**12** 12 12 13 13 14 14
02	**LITERATURE SURVEY** 2.1 GENERAL 2.2 EXISTING SYSTEM 2.3 TECHNOLOGICAL BACKGROUND 2.4 PROPOSED SYSTEM 2.5 RESEARCH GAP 2.6 SUMMARY	**15** 15 17 18 19 19 20
03	**METHODOLOGY** 3.1 SYSTEM REQUIREMENTS 3.2 PROPOSED SYSTEM ARCHITECTURE 3.3 DESCRIPTION OF MODULES 3.4 QUESTION BANK AND DATA PREPARATION 3.5 TEST ENGINE AND ADAPTIVE SELECTION 3.6 ELO ABILITY RATING 3.7 ANALYTICS AND AI MENTOR 3.8 READINESS MODEL 3.9 CODING JUDGE 3.10 SECURITY DESIGN 3.11 SUMMARY	**21** 23 24 24 25 26 27 28 29 30 32 33

04	**SYSTEM IMPLEMENTATION** 4.1 GENERAL 4.2 SYSTEM WORKFLOW 4.3 MODULE IMPLEMENTATION 4.4 BACKEND IMPLEMENTATION 4.5 DATABASE IMPLEMENTATION 4.6 USER INTERFACE 4.7 INTEGRATION 4.8 TESTING STRATEGY 4.9 SUMMARY	**36** 31 31 32 34 35 37 38 38 39
05	**EXPERIMENTAL ANALYSIS** 5.1 EXPERIMENTAL SETUP 5.2 PERFORMANCE EVALUATION 5.3 QUANTITATIVE RESULTS 5.4 VISUAL RESULTS 5.5 COMPARISON WITH EARLIER VERSION AND OTHER TOOLS 5.6 RESULT AND DISCUSSION 5.7 SUMMARY	**40** 41 41 42 43 45 52 53
06	**CONCLUSION AND FUTURE WORK** 6.1 CONCLUSION 6.2 LIMITATIONS AND CHALLENGES 6.3 FUTURE ENHANCEMENT 6.4 SUMMARY	**54** 54 54 55 55
07	**REFERENCES**	**57**
08	**SOURCE CODE**	**58**
09	**CONFERENCE APPLICATION DETAILS**	**63**
10	**CONFERENCE APPLICATION / SUBMISSION EVIDENCE**	**64**

VELS

INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMYUR, ADYAR - CHENNAI

A++ NAAC

CATEGORY 1 INSTITUTION BY UGC

EDUCOACH AI

An Adaptive AI-Powered Placement Preparation Platform using Multi-Agent Intelligence for Personalized Career Readiness

Author : Sathya Sri R • B.Tech CSE (AI & ML) | **Project Guide :** G. Sangeetha

Learn Prepare Get Placed

ABSTRACT

EduCoach AI is an adaptive AI-powered placement preparation platform that helps students prepare for campus recruitment through a personalized and intelligent learning experience. The platform uses a multi-agent system to provide technical, verbal, aptitude and coding practice with real-time performance analysis, weak-topic detection and adaptive recommendations. It ensures better readiness, builds confidence and improves placement chances for students.

OBJECTIVES

- Build a validated, explained question bank.
- Generate aptitude questions with worked solutions.
- Select 30-question tests with a per-topic Elo rating.
- Give feedback: review, notebook and a 7-day plan.
- Predict readiness and report the model honestly.
- Judge Python code and secure the application.

METHODOLOGY

1. **Build the bank.** 380 questions; 179 generated.
2. **Select.** 30 distinct: topic, subject, then category.
3. **Rate.** Elo ability per learner and topic. K = 20.
4. **Analyse.** Per-question accuracy: weak below 60%.
5. **Predict.** Random Forest, 8 features, after 3 tests.
6. **Guide.** Mentor advice, notebook, study plan.

APPLICATION INTERFACES



Figure 1: Home Dashboard (Placement Readiness & Practice Modules)

READINESS MODEL

Readiness model

Random Forest classifier (Low / Medium / High).

Simulated cohort of 3000 students (no real placement outcomes available). Replace it with real labelled placement outcomes before making real decisions.

HOLD-OUT ACCURACY	5-FOLD CV	MACRO F1	TRAIN / TEST SIZE
69.5%	71.5% ± 1.3	0.693	2250 / 750

Confusion matrix

actual \ predicted

	Low	Medium	High
Low	113	60	1
Medium	37	169	60
High	0	71	239

Feature importance

accuracy	0.295
avg_score	0.237
hard_accuracy	0.194
score_std	0.076
time_per_question	0.075
trend	0.068

Figure 2: Readiness Model - Performance & Confusion Matrix

RESULTS & DISCUSSION

- **380** Validated questions across 13 topics
- **30** Distinct questions in every timed test
- **24 / 24** Automated tests passed
- **69.5%** Readiness model hold-out accuracy

Category-wise Performance



Top Weak Topics (Example)

Topic	Accuracy	Level
Java	58%	Medium
DBMS	62%	Medium
Networks	65%	Medium
OOP	68%	Medium
Python	72%	High

Readiness Trend



Figure 3: Performance Analytics, Weak Topics & Readiness Trend

STANDARDS USED

- **IEEE 830 / ISO/IEC/IEEE 29148:** Software requirements.
- **IEEE 1016:** Software design description.
- **IEEE 829 / ISO/IEC/IEEE 29119:** Software testing.
- **ISO/IEC 25010:** Software product quality.
- **IEEE conference format:** paper and report.
- **Evaluation:** hold-out accuracy, 5-fold CV, macro F1.
- **Stack:** Flask, SQLite, scikit-learn, Chart.js.

CONCLUSION

EduCoach AI demonstrates that an adaptive, multi-agent based learning system can significantly improve placement preparation by personalizing the learning path, identifying weak topics and providing structured guidance. The system is ready for further enhancements with larger datasets, real placement outcomes and company-specific modules.

Smarter Learning Better Placements

ABSTRACT:

Placement preparation is still done with separate tools. Students practise coding on one website, aptitude on another and technical subjects from notes, and a mock test gives them a score without saying what to revise. These tools do not share any information about the learner, so practice is not matched to the learner's level and the overall readiness is never visible. This project proposes EduCoach AI, an adaptive placement-preparation platform that brings Technical, Verbal, Aptitude and coding practice into one web application with one learner profile. The platform holds 380 validated questions in 135 topics, of which 179 aptitude questions are generated automatically with correct answers and worked explanations. Every practice test has exactly 30 distinct questions under a countdown timer. An Elo-style ability rating is kept for every learner and topic and decides which questions are shown next. Every answer is stored, so accuracy, weak topics and strong topics are measured per question. After each test the student gets an answer review with explanations, and a mistake notebook, an AI Mentor with a seven-day plan and a Random Forest readiness prediction (Low, Medium or High) guide the next practice. Python coding problems are judged against hidden test cases. The application is protected with salted password hashing, CSRF tokens and login throttling, and an admin panel imports questions from CSV files. The platform is built with Flask, SQLite and scikit-learn. A suite of 24 automated tests passes, every topic gives a full 30-question test, the readiness model reaches 69.5% hold-out accuracy and 71.5% cross-validation accuracy, and the main pages respond in a few milliseconds. The model is trained on a simulated cohort because no real placement data was available, and this is stated openly in the application and in the report. The code judge is not a full sandbox and judges Python only. No study with real students has yet been carried out. Existing work usually treats adaptive rating, feedback and readiness prediction separately, and practice tools are usually built around one skill. The proposed system differs by carrying practice, adaptation, feedback and readiness estimation under a single learner profile that a college can host on an ordinary laptop.

Key Terms—Artificial Intelligence, Adaptive Learning, Machine Learning, Multi-Agent Systems, Personalized Learning, Placement Preparation, Student Performance Analysis, Career Readiness.

LIST OF FIGURES

NO.	TITLE	PAGE NO
Fig.3.1	Layered architecture of the EduCoach AI platform	21
Fig.3.2	Composition of the question bank by category and difficulty	24
Fig.3.3	Adaptive question-selection procedure	26
Fig.3.4	Elo expected-score curve used by the adaptive engine	28
Fig.4.1	Workflow of one practice test	33
Fig.4.2	Entity-relationship diagram of the SQLite database	36
Fig.5.1	Confusion matrix and feature importance of the readiness model	40
Fig.5.2	Login page	42
Fig.5.3	Student dashboard with readiness, average score and recommendation	43
Fig.5.4	Topic picker with difficulty selector and level badges	44
Fig.5.5	Timed 30-question test with countdown and progress bar	44
Fig.5.6	Result page with score ring, topic breakdown and answer review	45
Fig.5.7	Performance analytics: per-topic accuracy and ability rating	46
Fig.5.8	AI Mentor insights and 7-day study plan	46
Fig.5.9	Mistake notebook	47
Fig.5.10	Readiness-model page with evaluation metrics	47
Fig.5.11	Coding judge showing the verdict for a submission	48
Fig.5.12	Admin panel with question statistics and CSV import	48
Fig.8.1	Student dashboard with readiness, average score and recommendation	61
Fig.8.2	Readiness-model page with evaluation metrics	61

LIST OF TABLES

NO.	TITLE	PAGE NO
Table 1.1	Conventional placement preparation compared with the proposed system	11
Table 2.1	Summary of the surveyed literature	15
Table 2.2	Advantages and disadvantages of the surveyed works and the addition made by the proposed system	16
Table 3.1	Hardware requirements	19
Table 3.2	Software requirements	20
Table 3.3	Composition of the question bank	24
Table 3.4	Aptitude topics with generated questions	25
Table 3.5	Ability-rating levels	28
Table 3.6	Features of the readiness model	29
Table 3.7	Security threats and the controls used	31
Table 4.1	Main routes of the Flask application	35
Table 4.2	Main database entities	36
Table 4.3	Application views and their purpose	37
Table 4.4	Module-wise implementation status	38
Table 4.5	Automated test suite	38
Table 5.1	Evaluation measures and what they show	40
Table 5.2	Performance evaluation of the proposed system	40
Table 5.3	Response time of the main operations	41
Table 5.4	Comparison with the earlier version of the project	49
Table 5.5	Coverage of typical preparation tools and the proposed system	49
Table 5.6	Results and the limiting factors reported for each module	51

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
AI	Artificial Intelligence
API	Application Programming Interface
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DBMS	Database Management System
ER	Entity–Relationship
F1	F1 Score (Harmonic Mean of Precision and Recall)
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IRT	Item Response Theory
IoT	Internet of Things
JSON	JavaScript Object Notation
ML	Machine Learning
OWASP	Open Worldwide Application Security Project
RF	Random Forest
SQL	Structured Query Language
UI	User Interface

CHAPTER 1

1. INTRODUCTION:

1.1 GENERAL

Campus placements are one of the most important milestones for an engineering student. Most recruiters test candidates in three stages: an online screening round of aptitude, verbal and technical questions, a coding round, and finally interviews. A student therefore has to prepare for several different skills at the same time, and has to decide every day what to practise next.

In current practice this preparation is spread over many separate tools. Coding problems are practised on one website, aptitude questions are taken from a printed book or another website, and technical subjects such as DBMS or operating systems are revised from class notes. Each tool has improved on its own, but they do not share any information about the learner. A student can finish a mock test with a score of 60% and still not know which topic caused the loss of marks, which questions were answered wrongly, or whether the preparation is improving.

What is missing is a single learner model that follows the student across all the practice areas and ends in clear guidance. This project builds that model into a web platform called EduCoach AI. The platform has three central ideas. The first is an ability rating for every topic, updated after every answered question, on which the choice of the next questions depends. The second is per-question analytics, so that weak and strong topics are found from what the student actually answered and not only from test totals. The third is a guidance layer, made of a mistake notebook, a rule-based AI Mentor with a seven-day plan, and a machine-learning model that predicts placement readiness.

Three practical problems follow from the conventional arrangement. The first is relevance: a fixed mock test gives the same difficulty to every student, so strong students are not challenged and weak students are discouraged. The second is feedback: a bare score does not say what to revise. The third is fragmentation: aptitude, verbal, technical and coding practice are kept in different places, so no overall picture of readiness exists.

The platform is built with Flask, SQLite and scikit-learn. It currently holds 380 questions in 135 topics of three categories (Technical, Verbal and Aptitude), plus 13 auto-judged coding problems. Every practice test has exactly 30 distinct questions and a countdown timer. The setting the system is designed for is a college training and placement cell, or an individual student preparing alone, who needs a private and self-hosted tool that runs on an ordinary laptop.

1.2 PROBLEM STATEMENT

Traditional placement preparation depends on static question sets and on separate, unconnected tools. The cost is practice that is not matched to the learner's level, feedback that stops at a score, and no combined view of readiness across aptitude, verbal, technical and coding skills.

The literature reviewed for this project treats adaptive learning, feedback and readiness prediction as separate research topics, and the commercial practice tools that students commonly use are usually built around one skill, such as coding. This project closes the gap by developing one web platform that practises all four skill areas, adapts the questions to a per-topic ability rating, gives explanation-based feedback after every test, and estimates readiness from the practice history.

Table 1.1 contrasts the conventional arrangement with the proposed one. The comparison is qualitative

Table 1.1. Conventional truck monitoring compared with the proposed system

Aspect	Conventional preparation	Proposed system (EduCoach AI)
Practice areas	Several separate websites and books	Technical, Verbal, Aptitude and Coding in one platform
Question selection	Fixed mock tests, same for everyone	Adaptive selection near the learner's per-topic ability rating
Test format	Varies; often untimed practice	30 distinct questions, countdown timer, auto-submit
Feedback	Total score, sometimes correct answers	Answer review with explanations and a mistake notebook
Weak-topic detection	Done by the student, by memory	Computed from per-question accuracy of every topic
Guidance	Self-planned	AI Mentor with trend analysis and a 7-day plan
Readiness estimate	None, or a guess from mock scores	Random Forest prediction (Low / Medium / High) with confidence
Coding practice	Separate online judge	Built-in judge with hidden test cases for Python
Question supply	Limited to what is available	Curated questions, generated aptitude questions and CSV import
Security	Not applicable	Salted password hashes, CSRF protection, login throttling

1.3 OBJECTIVES

The project develops a web platform backed by a database, an adaptive engine and a small machine-learning component. The specific objectives are listed below

1. Build a question bank for Technical, Verbal and Aptitude practice in which every question is distinct, has four options, a correct answer and an explanation.
2. Generate aptitude questions automatically, with correct answers and worked solutions, so that the bank can grow without manual effort.
3. Design a timed test engine that delivers exactly 30 distinct questions per test and never repeats a question inside a test.
4. Implement an Elo-style ability rating for every learner and topic, and use it to choose questions of suitable difficulty.
5. Record every answer so that accuracy, weak topics and strong topics are measured per question and per topic.
6. Provide feedback after every test: score, topic breakdown, explanations and a notebook of past mistakes.
7. Build an AI Mentor that explains the learner's trend and produces a personalised seven-day study plan.
8. Train and evaluate a Random Forest model that predicts placement readiness, and report its evaluation honestly.
9. Provide an auto-judged coding module with hidden test cases and clear verdicts.
10. Secure the application (password hashing, CSRF protection, login throttling) and provide an admin panel for managing the question bank.

1.4 SCOPE AND MODULES

The system is organised as a 12-module platform arranged in four layers. The first layer, presentation, contains the web interface. The second layer, application, holds authentication and security, the test engine, the adaptive selector, analytics, the AI Mentor and the admin panel. The third layer, intelligence and judge, contains the Elo ability rating, the question generators, the readiness model and the code judge. The fourth layer is the SQLite database.

The platform is evaluated by an automated test suite of 24 tests, by measurements of the question bank, by hold-out evaluation of the readiness model, and by response-time measurements. The scope has limits that are stated openly in later chapters. The readiness model is trained on a simulated cohort because no real placement-outcome data was available, the code judge is not a

full security sandbox, only Python submissions are judged automatically, and no study with real students has yet been carried out.

In scope are the 12 modules named above, their integration into one web application, and the evaluation described in Chapter 5. Out of scope are a classroom-scale user study, training the model on real placement outcomes, multi-language judging, and deployment for many simultaneous users.

1.5 ORGANISATION OF THE REPORT

The report has eight chapters. Chapter 1 introduces the problem, the objectives and the scope. Chapter 2 reviews the literature, summarises the technologies the system builds on, and states the research gap. Chapter 3 presents the methodology: requirements, architecture, the modules, the question bank, the adaptive engine, the readiness model, the code judge and the security design. Chapter 4 describes the implementation, including the workflow, backend, database, views, integration status and testing. Chapter 5 reports the experimental set-up, the quantitative results, the visual results and a comparison with the earlier version and with typical preparation tools. Chapter 6 gives the conclusion, the limitations and future work. Chapter 7 lists the references, and Chapter 8 describes the source code

1.6 SUMMARY

This chapter described how placement preparation is carried out today and why separate tools leave gaps in relevance, feedback and overall guidance. It stated the problem, listed ten objectives, and outlined the 12-module, four-layer platform that the rest of the report describes. Chapter 2 reviews the published work that this project builds on and identifies the gap it addresses.

CHAPTER 2

2. LITERATURE SURVEY:

2.1 GENERAL

Eight works were studied for this project. They cover adaptive rating of learners, knowledge tracing, machine-learning classification, and the educational evidence for testing and feedback. Each was summarised, and its advantages and disadvantages were compared with the proposed work. The surveyed works fall into two groups. The first group, adaptive modelling of learners, describes how a system can estimate what a learner knows. The second group, learning-science studies, explains why frequent testing and good feedback improve learning, which is the motivation for the design of the platform.

None of the surveyed works builds a complete placement-preparation platform that combines several skill areas, a per-topic ability rating, answer-level analytics, a coding judge and a readiness prediction in one self-hosted application. The practice tools that students commonly use are usually organised around a single skill, most often coding, and offer fixed question lists rather than an adaptive rating.

2.2 EXISTING SYSTEM

Pelánek [1] reviews applications of the Elo rating system in adaptive educational systems. The Elo system was designed for rating chess players [2], and it is simple, updates after every answer and needs no complicated parameter fitting. Pelánek discusses how it can estimate learner skill and item difficulty together. The work is a review of the method and does not describe a complete practice platform, which is the extension made by the proposed system: the same rating idea is used inside a platform with tests, analytics, mentoring and coding practice.

Corbett and Anderson [3] introduced knowledge tracing, a Bayesian model that follows the probability that a learner knows a skill as the learner answers questions in an intelligent tutoring system for programming. The model gives a principled estimate of mastery, but it needs the skills to be defined in advance and its parameters to be estimated for every skill. Piech et al. [4] extended the idea with recurrent neural networks, called deep knowledge tracing. This approach can capture richer patterns but needs a large amount of interaction data from real learners, which a new platform does not have.

Breiman [5] introduced random forests, an ensemble of decision trees trained on random samples of the data and features. Random forests give good accuracy with little tuning, handle features of different scales and report feature importance, which makes them suitable for the readiness classifier of this project. The scikit-learn library [6] provides the implementation used here.

Roediger and Karpicke [7] showed that taking a test improves long-term retention more than simply studying the material again, an effect called test-enhanced learning. This supports the central design choice of the platform, which is frequent practice through tests. Hattie and Timperley [8] analysed the effect of feedback on learning and concluded that feedback is most useful when it tells the learner where they are going, how they are doing and what to do next. This is the model for the answer review, mistake notebook and AI Mentor. Black and Wiliam [9] reviewed formative assessment, in which assessment is used to guide teaching and learning, and found that it raises achievement. Lord [10] describes item response theory, the statistical foundation of adaptive testing, in which every question has a difficulty and every learner an ability on the same scale.

Finally, web application security guidance from the OWASP Top Ten [11] lists broken authentication, cross-site request forgery and weak cryptographic storage among the common risks, and these informed the security design of the platform. Grinberg [12] describes the Flask framework, whose small core and extensions suit an application like this one
Table 2.1 summarises all the works.

Table 2.1. Summary of the surveyed literature

Ref.	Work	Summary	Limitation for this problem
1	Pelánek, Applications of the Elo rating system in adaptive educational systems, 2016 [1]	Reviews use of Elo rating to estimate learner skill and item difficulty online.	A method review; no complete practice platform, analytics or mentoring.
3	Corbett and Anderson, Knowledge tracing, 1994 [3]	Bayesian estimate of skill mastery in a programming tutor.	Needs predefined skills and per-skill parameters; single tutor domain.
4	Piech et al., Deep Knowledge Tracing, 2015 [4]	Recurrent neural network predicts future answers from past answers.	Needs large real interaction logs; hard to explain to learners.
5	Breiman, Random Forests, 2001 [5]	Ensemble of decision trees with feature importance.	A general algorithm; needs labelled data for the specific task.

Ref.	Work	Summary	Limitation for this problem
7	Roediger and Karpicke, Test-enhanced learning, 2006 [7]	Testing improves long-term retention compared with restudying.	Educational evidence only; no software design.
8	Hattie and Timperley, The power of feedback, 2007 [8]	Feedback works best when it says where to go, how it is going and what next.	Educational evidence only; no automated implementation.
9	Black and Wiliam, Assessment and classroom learning, 1998 [9]	Formative assessment raises achievement.	Classroom study; not about self-paced online practice.
-	Typical coding and aptitude practice websites (general observation)	Large question lists; online judges for coding.	Usually one skill per tool; fixed lists; limited learner model.

Comparative analysis

Table 2.2 sets the advantage of each surveyed work against its disadvantage and states what the proposed system adds.

Table 2.2. Advantages and disadvantages of the surveyed works and the addition made by the proposed system

Work	Advantage	Disadvantage	Addition in the proposed system
Elo rating [1], [2]	Simple, online, no heavy fitting	Method only	Per-topic Elo ratings drive question choice inside a full platform
Knowledge tracing [3]	Principled mastery estimate	Predefined skills, parameters per skill	Uses a lightweight rating that works from the first answer
Deep knowledge tracing [4]	Captures rich patterns	Needs large data	Avoids the data requirement; keeps explanations simple
Random forest [5], [6]	Accurate, robust, interpretable importances	Needs labelled data	Trained on a simulated cohort with honest evaluation, replaceable by real data
Test-enhanced learning [7]	Evidence that tests aid retention	No software	Frequent timed tests of 30 questions
Feedback research [8], [9]	Evidence for useful feedback	No software	Answer review, mistake notebook and AI Mentor plan

Work	Advantage	Disadvantage	Addition in the proposed system
Typical practice sites	Large question lists	One skill; fixed lists	Four skill areas in one adaptive platform

2.3 TECHNOLOGICAL BACKGROUND

The works surveyed above rest on a small set of techniques. This section summarises the ones the proposed system uses, so that the design choices in Chapter 3 can be read in context.

A. Adaptive Testing and Rating Systems

In adaptive testing the questions are chosen according to the answers given so far. Item response theory [10] places learner ability and question difficulty on one scale, so that the probability of a correct answer depends on the difference between them. The Elo rating system [2] uses the same idea in a simpler form: after every answer the learner's rating moves up if the learner did better than expected and down if worse, by an amount that depends on how surprising the result was. Because the update needs only the current rating and the question difficulty, it can run immediately after every answer on a small system.

B. Knowledge Tracing

Knowledge tracing [3], [4] follows the state of a learner's knowledge over time. Bayesian and neural versions exist. They are powerful but depend on large amounts of data or on carefully defined skill models. For a new platform with no historical data, a simple rating is the practical starting point, and the design leaves room to replace it later.

C. Random Forest Classification

A random forest [5] trains many decision trees on bootstrap samples of the training data, each tree using a random subset of the features, and combines their votes. Averaging reduces over-fitting, and the fraction of trees voting for a class gives a confidence value. The importance of each feature can be read from the trained forest, which is useful for explaining the model to the learner.

D. Testing and Feedback

The educational studies [7], [8], [9] provide the pedagogical basis. Retrieval through testing strengthens memory, and feedback is most effective when it is specific. The platform therefore

reviews every answer with an explanation, collects wrong answers in a notebook, and turns the learner's accuracy data into a concrete plan.

E. Web Technologies

Flask [12] is a lightweight Python web framework, well suited to an application whose main job is to serve pages, apply rules and talk to a database. Jinja2 renders the HTML templates and escapes user content automatically. SQLite is a serverless relational database stored in one file, which makes the platform easy to install and back up. Chart.js draws the charts in the browser.

F. Application Security

Passwords must be stored as salted slow hashes, never as plain text or fast unsalted hashes. State-changing requests must carry a secret token so that other websites cannot submit them on behalf of a logged-in student (cross-site request forgery). Repeated failed logins should be limited. Running code written by users needs isolation, because such code could be harmful [11].

2.4 PROPOSED SYSTEM

The proposed system is a single web platform in which one learner model connects every practice activity. The student chooses a category and topic. The test engine selects 30 distinct questions near the learner's rating for that topic, and the test runs under a countdown timer. Each answer is stored, and the Elo rating of the topic of every question is updated. Analytics, the mistake notebook and the AI Mentor read the stored answers. After three tests the readiness model predicts Low, Medium or High readiness with a confidence value. Coding problems are judged automatically against hidden test cases.

The platform is therefore more than a question bank or a stand-alone mock test. It integrates practice, adaptation, feedback and readiness estimation under one learner profile.

2.5 RESEARCH GAP

The surveyed works and the commonly used practice tools do not combine all of the following in one self-hosted platform: adaptive selection by a per-topic rating, practice across aptitude, verbal, technical and coding skills, answer-level analytics with feedback, and a readiness estimate. Four specific shortcomings follow from the review.

1. Rating and knowledge-tracing research describes methods, but not a complete practice platform that a college can install and use.

2. Practice tools are usually organised around one skill, so the learner's overall readiness across skills is never shown.
3. Feedback is often limited to a score, although the feedback research [8] shows that specific guidance on what to do next is what helps.
4. Readiness estimates, where they exist, are rarely explained or evaluated openly, and the data they rely on is often not stated.

2.6 SUMMARY

This chapter reviewed eight works on adaptive rating, knowledge tracing, random forests, testing and feedback. They show that each ingredient is well understood but that none of them delivers a complete preparation platform. The proposed system is introduced as a response to this gap, and Chapter 3 describes its design.

CHAPTER 3

3. METHODOLOGY:

3.1 SYSTEM REQUIREMENTS

EduCoach AI is a web application with a small machine-learning component, so its requirements are modest. It runs on an ordinary laptop, and students need only a web browser. Tables 3.1 and 3.2 list the hardware and software used.

A. Hardware Requirements

Table 3.1. Hardware requirements

Component	Specification
Server / host machine	Laptop or desktop, dual-core CPU or better
Memory (RAM)	4 GB minimum, 8 GB recommended
Storage	200 MB free space (the SQLite database is a single file)
Client device	Any device with a modern web browser (desktop, tablet or phone)
Network	Local network or localhost; internet only for the Chart.js library

B. Software Requirements

Table 3.2. Software requirements

Category	Tools / technologies
Programming language	Python 3.11 or newer (backend, adaptive engine, model); HTML, CSS and JavaScript (interface)
Web framework	Flask 3.x with Jinja2 templates and Werkzeug security utilities
Database	SQLite 3 (Python standard library)
Machine learning	scikit-learn (RandomForestClassifier), NumPy
Charts	Chart.js (loaded in the browser)
Code judge	Python subprocess module with timeout and POSIX resource limits
Testing	Python unittest (24 automated tests)
Development tools / OS	VS Code, Git; Windows 10/11, Ubuntu or macOS

Each requirement follows from how the platform works. SQLite was chosen because a single-file database needs no installation and is enough for one classroom of students. Python hosts the web

framework, the question generators, the adaptive engine, the Random Forest model and the judge, which avoids connecting several languages. No GPU is needed because the only learned model is a small Random Forest that trains in about two seconds.

3.2 PROPOSED SYSTEM ARCHITECTURE

A. Architecture Diagram

The system is structured as a 12-module platform in four layers: presentation, application, intelligence and judge, and data (Fig.3.1). A request travels from the browser at the top to the database at the bottom and back.

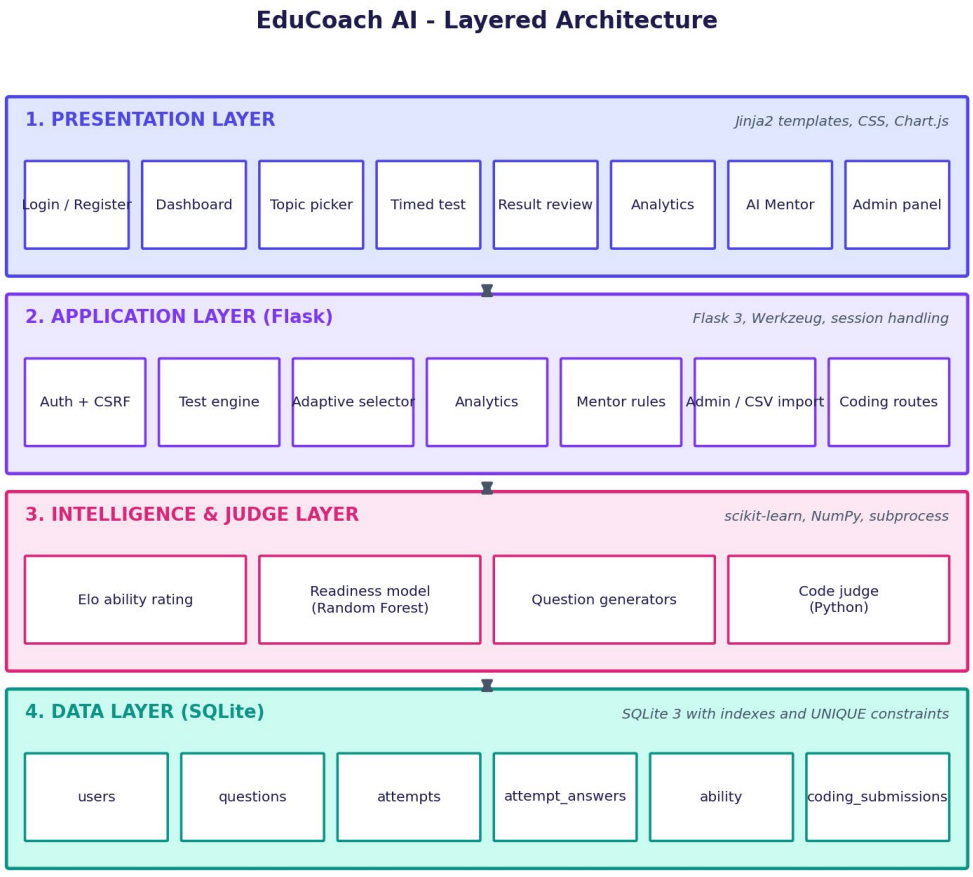


Fig.3.1. Layered architecture of the EduCoach AI platform

B. Proposed Methodology

A student registers and logs in. The student picks a category (Technical, Verbal, Aptitude or Coding), a topic and a difficulty mode. For practice tests, the test engine selects 30 distinct questions using the adaptive selector described in Section 3.5, and the browser shows them under

a countdown timer. When the student submits, or when the timer ends, the server scores the answers, stores the attempt and every individual answer, and updates the Elo ability rating of the topic of each question.

From the stored answers the application computes accuracy per topic, finds weak and strong topics, builds the mistake notebook and writes the AI Mentor advice. Once three tests exist, the readiness model converts the attempt history into eight features and predicts Low, Medium or High readiness. Coding problems follow a separate path: the submitted program is run against test cases by the judge and the verdict is stored.

C. System Design

The design separates three kinds of work. Request handling (login, pages, forms) is done by Flask routes. Decision logic (question selection, rating update, analytics rules, mentor rules) is written as small Python functions that do not depend on the web framework, so they can be tested alone. Storage is handled in one place, through SQLite queries. This separation allowed the 24 automated tests to call the adaptive functions directly.

Three actors use the system. The student takes tests, reviews results and solves coding problems. The administrator, who is the first registered user or the user named by the ADMIN_EMAIL setting, imports questions and watches platform statistics. The system itself runs the selection, scoring, rating and analytics automatically.

D. Layer-wise Description

The presentation layer is a set of 14 Jinja2 templates with one stylesheet. It shows the dashboard, topic picker, test, result, analytics, mentor, mistake notebook, model, coding and admin pages. The only script that runs in the browser besides Chart.js is the countdown timer and progress bar of the test page.

The application layer is the Flask application. It authenticates users, protects every form with a CSRF token, serves the pages, and applies the business rules. All rules that affect a decision, such as the number of questions in a test or the number of answers needed before a topic is called weak, are named constants at the top of the file.

The intelligence and judge layer holds the parts that learn or evaluate: the Elo rating, the aptitude question generators, the readiness model and the code judge. The data layer is the SQLite database with six tables, described in Chapter 4.

3.3 DESCRIPTION OF MODULES

A. Authentication and Security
Registers students, validates the email and password, stores a salted password hash, logs users in and out, limits repeated failed logins, and protects all forms with CSRF tokens. The first registered account becomes the administrator.

B. Question Bank and Generators
Holds 380 questions with options, correct answer, difficulty and explanation. Aptitude questions for 15 topics are generated with correct answers and worked solutions. The bank is seeded automatically and can be extended by CSV import.

C. Test Engine
Selects exactly 30 distinct questions, shows them under a countdown timer (60 seconds per question for Technical and Verbal tests, 90 seconds for Aptitude), scores the submission and stores the attempt together with every answer.

D. Adaptive Engine (Elo)
Keeps one ability rating per learner and topic. It supplies the target difficulty for question selection and is updated after every answered question.

E. Analytics
Computes per-topic accuracy from stored answers, lists weak topics (below 60%) and strong topics (80% or more) that have at least three answers, and prepares the data for the charts.

F. Readiness Model
A Random Forest classifier that turns the learner's attempt history into a Low, Medium or High readiness label with a confidence value. It is described in Section 3.8.

G. AI Mentor

A rule-based advisor that states the learner's trend, names weak and strong topics with suggested difficulty, and builds a seven-day plan. It is called an AI Mentor because its advice is computed from the learner's own data, using the rating, the analytics and the model.

H. Mistake Notebook
Lists the questions the learner answered wrongly, most recent first, with the learner's choice, the correct answer and the explanation.

I. Coding Judge
Provides 13 problems with 56 test cases in total. A Python submission is run for each test case in a separate process with a time limit, and the output is compared with the expected output.

J. Admin Panel

Shows the number of users, tests and code submissions, and the questions per category. It imports questions from a CSV file with validation and skips duplicates.

K. Database

Stores users, questions, attempts, individual answers, ability ratings and coding submissions in SQLite with indexes and uniqueness constraints.

L. Web Interface

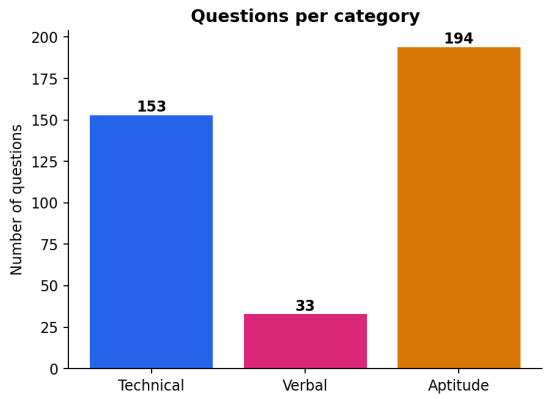
Presents all of the above in a responsive, colour-coded interface that works on phones and desktops.

3.4 QUESTION BANK AND DATA PREPARATION

The platform has no external dataset. Its data is the question bank, which has three sources. The first is a set of original starter questions, one for each topic of the catalogue. The second is a set of curated questions written and reviewed for this project, with explanations. The third is a set of generated aptitude questions. In total the bank holds 380 questions, as shown in Table 3.3 and Fig.3.2.

Table 3.3. Composition of the question bank

Category	Topics	Questions	Source of questions
Technical	90 (in 12 subjects)	153	Starter and curated questions
Verbal	15	33	Starter and curated questions
Aptitude	30	194	179 generated (15 topics) plus starter questions for the other topics
Total	135	380	Difficulty: 183 Easy, 119 Medium, 78 Hard



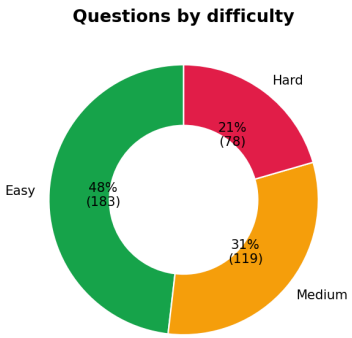


Fig.3.2. Composition of the question bank by category and difficulty

A. Generated aptitude questions

Aptitude questions are produced by small generator functions, one for each of 15 topics. A generator takes a difficulty level (Easy, Medium or Hard) and a random number generator and returns the question text, four options, the correct answer and a worked explanation. The numbers in the question are drawn so that the answer is always exact, and three distractor options are placed near the correct value. The random generator is seeded with the topic name, so the same bank is produced every time and the tests are reproducible. Four questions per difficulty level are generated for each topic, giving 179 questions.

Table 3.4. Aptitude topics with generated questions

Group	Topics with generators
Arithmetic	Percentages, Profit & Loss, Simple Interest, Compound Interest, Averages
Ratio and work	Ratio & Proportion, Time & Work, Time Speed & Distance, LCM & HCF
Numbers and algebra	Number Series, Problems on Ages, Algebra
Geometry and counting	Mensuration, Probability, Permutation & Combination

A representative generator is shown below. It produces a percentage question at each of the three levels and builds the explanation from the same numbers as the answer, so the explanation cannot disagree with the answer.

B. Validation of the bank

Every question, whatever its source, must pass four checks: it has exactly four distinct options, the correct answer is one of the options, the difficulty is Easy, Medium or Hard, and the combination of subject, topic and question text is unique. The checks run when the bank is seeded, when a CSV file is imported, and in the automated test suite, which also verifies that every topic of the catalogue has at least one question.

3.5 TEST ENGINE AND ADAPTIVE SECTION

Every practice test has exactly 30 distinct questions, a value held in one constant. Questions are chosen from three pools in order: the questions of the chosen topic, then the other questions of the same subject, then the other questions of the same category. The pools are used one after another until 30 questions have been chosen, so a topic with only a few questions of its own still gives a full test, and the test page tells the student when related questions were added. Fig.3.3 shows the procedure.

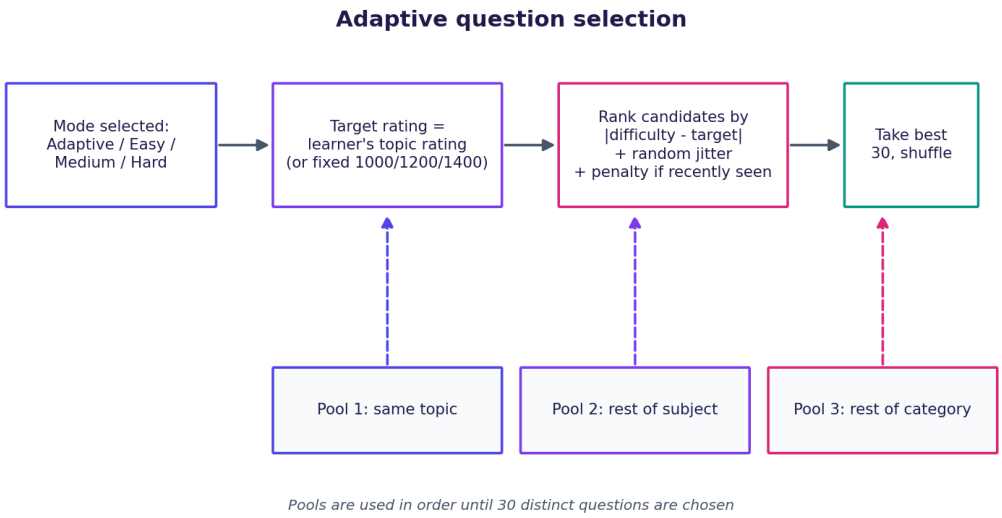


Fig.3.3. Adaptive question-selection procedure

Inside a pool, candidates are ranked by a score that is small for questions whose difficulty is close to the target. The target depends on the mode. In Adaptive mode it is the learner's current rating for the topic. In Easy, Medium and Hard modes it is fixed at 1000, 1200 and 1400. A random jitter is added so that two tests on the same topic are not identical, and a penalty is added to questions that the learner answered in the last 60 answers, so that new questions are preferred. The selected questions are shuffled before they are shown. The core of the function is reproduced below.

3.6 ELO ABILITY RATING

Each learner has one rating per topic, starting at 1100. Each question has a fixed difficulty rating: 1000 for Easy, 1200 for Medium and 1400 for Hard. After every answered question the rating of the question's topic is updated with the Elo rule [2], [1]. If the learner's rating is R and the question's rating is Q, the expected probability of a correct answer is

$$E = 1 / (1 + 10^{((Q - R) / 400)})$$

and the new rating, after a result S that is 1 for a correct answer and 0 for a wrong or missing answer, is

$$R' = R + K\,(S - E), \qquad K = 20$$

A correct answer to a hard question raises the rating more than a correct answer to an easy one, and a wrong answer to an easy question lowers it more than a wrong answer to a hard one. Fig.3.4 shows the expected-score curve. The rating is converted into four levels, shown in Table 3.5, which appear next to each topic in the interface.

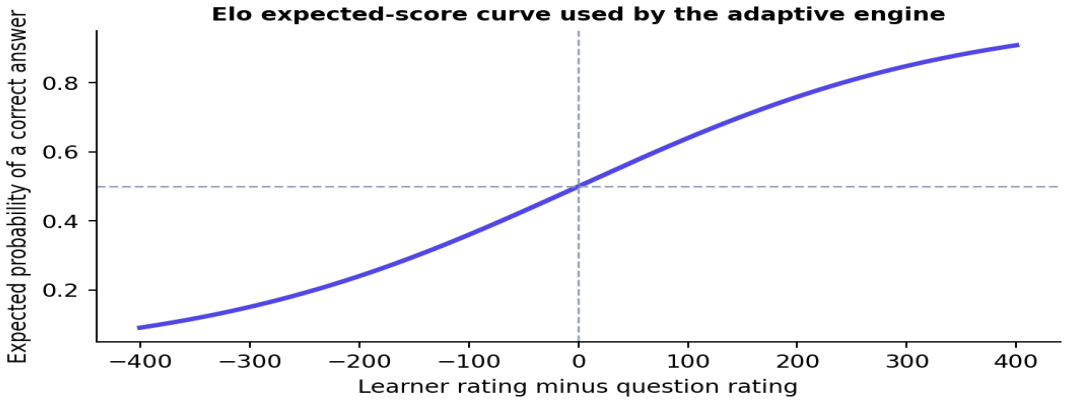


Fig.3.4. Elo expected-score curve used by the adaptive engine

Table 3.5. Ability-rating levels

Rating	Level	Meaning in the interface
Below 1050	Beginner	Practise Easy questions first
1050 to 1199	Intermediate	Mixed Easy and Medium practice
1200 to 1349	Advanced	Medium and Hard questions suit the learner
1350 and above	Expert	Hard questions to stay challenged

3.7 ANALYTICS AND AI MENTOR

The Accuracy is measured per answered question and grouped by the topic of the question, not by the topic the student chose when starting the test. This matters because a test may include related topics. A topic is called weak when its accuracy is below 60% and strong when it is 80% or higher, but only after at least three answers, so that one lucky or unlucky answer does not label a topic.

The AI Mentor combines four sources: the readiness label, the trend of the last six tests, the weak and strong topics, and the ability level of each weak topic. The trend compares the average of the latest three tests with the average of the earlier ones, and reports improving, declining or steady when the change is more than five percentage points. For each weak topic the mentor suggests Easy difficulty when the learner is a Beginner or Intermediate and Medium difficulty otherwise. The seven-day plan alternates focused practice on the weakest topic, review of mistakes, a timed Aptitude test, a Technical test on a new topic, coding practice and a final mixed review. The recommended-next card on the dashboard picks the weakest topic and starts a test on it directly.

3.8 READLINESS MODEL

The readiness model predicts one of three labels, Low, Medium or High, from the learner's history. It becomes available after three tests.

A. Features

Table 3.6. Features of the readiness model

Feature	Meaning
Accuracy	Mean fraction of correct answers over all tests
avg_score	Mean test score in percent
time_per_question	Mean seconds spent per question
score_std	Standard deviation of test scores (consistency)
Trend	Change between the first and the latest tests, divided by 100
tests_taken	Number of tests completed
hard_accuracy	Mean accuracy in tests taken at Hard difficulty (estimated from overall accuracy if none)
topics_covered	Number of distinct topics answered

B. Training data

There is no dataset of real students with known placement outcomes. The model is therefore trained on a simulated cohort of 3,000 students, and this limitation is stated on the model page of the application and in this report. In the simulation, each student has a hidden skill value drawn from a normal distribution. Accuracy, speed, consistency, trend and the other features are generated from the skill with random noise, and the readiness label is derived from the skill plus further noise. The

label is therefore not a simple threshold on a single feature, which was the weakness of the first version of this project, where the label was a direct function of the score.

C. Model and evaluation

A Random Forest of 200 trees with a maximum depth of 8 and at least 5 samples per leaf is trained on 2250 students and evaluated on 750 held-out students, with a stratified split. Five-fold cross-validation on the full cohort is also reported. The evaluation reports accuracy, macro F1 score, the confusion matrix and the feature importances, which are shown in Chapter 5. The purpose of the model in this project is to demonstrate a complete and honestly evaluated pipeline that can be retrained on real labelled data when it becomes available

3.9 CODING JUDGE

The coding module offers problems that read from standard input and write to standard output. Each problem has a statement, a sample, and several test cases, of which the first two are shown to the student and the rest are hidden. When a Python submission is received, the judge writes it to a temporary file and runs it once for each test case in a separate Python process in isolated mode, with a time limit of three seconds and, on Linux and macOS, a limit on memory and CPU time.

The output is compared with the expected output after removing surrounding whitespace. The verdict is Accepted when all tests pass, and otherwise the first failure type: Wrong Answer, Runtime Error or Time Limit Exceeded. Submissions in other languages are stored, but are not judged automatically. The judge is a teaching aid and not a complete security sandbox, and this is stated in the documentation.

3.10 SECURITY DESIGN

The first version of the project stored passwords with an unsalted fast hash, used a fixed secret key in the source code and ran with debug mode on. The second version addresses these and other risks, following the guidance in [11].

Table 3.7. Security threats and the controls used

Threat	Control in EduCoach AI
Stolen password database	Passwords stored as salted hashes with Werkzeug generate_password_hash; never reversible
Weak passwords	Minimum 8 characters with letters and a digit; valid email required

Threat	Control in EduCoach AI
Cross-site request forgery	Random token in the session; every POST must send the matching token, otherwise HTTP 400
Password guessing	After 5 failed logins for the same email and address, login is blocked for 5 minutes
Session theft and fixation	Session cleared at login; cookies are HTTP-only and SameSite=Lax; secret key generated per installation
Unauthorised admin access	Admin pages check an administrator flag, otherwise HTTP 403
Cross-site scripting	Jinja2 escapes all content automatically
SQL injection	All database queries use bound parameters
Malicious uploads	Only .csv files, limited to 1 MB, parsed as text and validated row by row
Hostile submitted code	Separate process, time and memory limits, 20,000-character limit; documented as not a full sandbox

3.1 SUMMARY

This chapter set out the hardware and software requirements, the four-layer architecture and the role of each of the 12 modules. It then described the question bank and its generators, the adaptive selection of 30 questions, the Elo rating, the analytics and mentor rules, the readiness model, the coding judge and the security design. Chapter 4 explains how these parts were implemented and joined together.

CHAPTER 4

4. SYSTEM IMPLEMENTATION:

4.1 GENERAL

The implementation turns the architecture of Chapter 3 into a working application. The whole backend is written in Python with Flask. The decision logic is kept in small functions, the question bank is built by a separate module, the readiness model and the judge are separate modules, and the interface is a set of Jinja2 templates with one stylesheet. The challenge is less in any single module than in keeping them independent enough to test, while sharing one learner profile.

The technology choices follow the requirement tables of Chapter 3. Flask and SQLite form the backend, scikit-learn provides the Random Forest, and Chart.js draws the charts in the browser. The following sections describe how each part was put together.

4.2 SYSTEM WORKFLOW

Fig.4.1 shows the processing of one practice test. The student logs in and chooses a topic and a mode. The test engine selects 30 distinct questions and the test page is shown with a countdown. On submission, the answers are scored, the attempt and each answer are stored, and the Elo rating is updated. The result page then shows the review. All other pages read the stored answers.

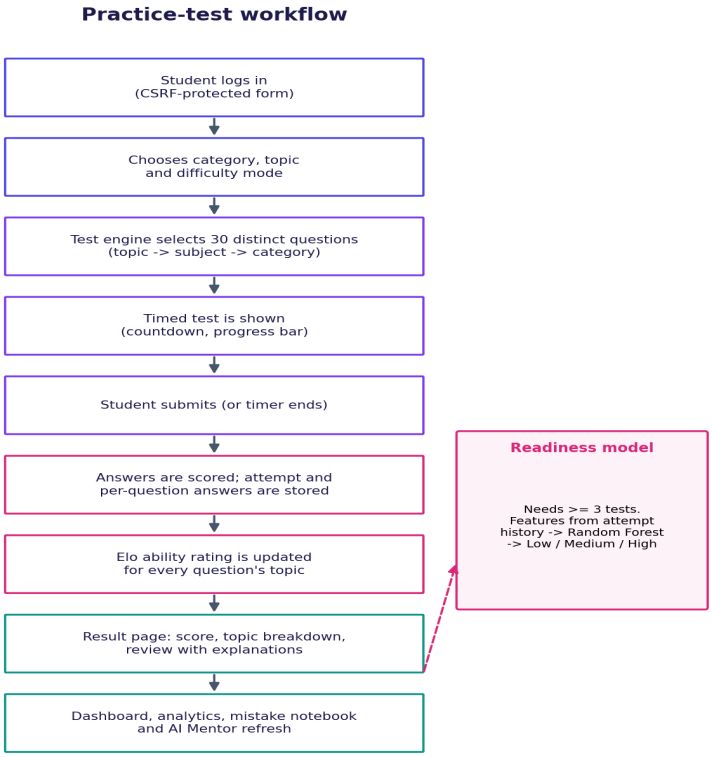


Fig.4.1. workflow of O\one practice test

Step-by-step description of the workflow

In the first step the student authenticates, and the server stores the user identifier in the session. In the second step the student sends a POST request that names the category, subject, topic and difficulty mode; the request is accepted only with a valid CSRF token. In the third step the test engine selects the questions and keeps their identifiers, the start time and the allowed time in the session.

In the fourth step the test page is rendered with the questions in a random order and a countdown that submits the form automatically at zero. In the fifth step the server receives the answers, compares each with the stored correct answer, and computes the score. In the sixth step it writes one row to the attempts table and one row for every question to the answers table, and updates the ratings. In the seventh step the student is redirected to the result page, which can be opened again later from the history list.

4.3 MODULE IMPLEMENTATION

A. Question bank builder

The module builds the whole bank as a list of dictionaries. It converts the original one-per-topic questions, adds the curated questions, calls the generators for the aptitude topics and shuffles the order of options with a seed derived from the question, so that the correct answer is not always in the same position. On start-up the application inserts the questions into SQLite with INSERT OR IGNORE, which makes seeding repeatable and prevents duplicates.

B. Test engine and adaptive selector

The selector queries the three pools, ranks the candidates, and returns identifiers. It is a plain function that receives a database connection, which allowed it to be tested directly, for example to check that a learner with a very high rating receives only Hard questions and that no test has more than 30 questions.

C. Rating, analytics and mentor

After submission the rating update is applied in one loop over the answered questions and written with an upsert statement. Analytics are SQL aggregations over the answers table joined with the questions table. The mentor is a function that builds a list of advice strings and a seven-day plan from these aggregates.

D. Readiness model

The model is trained once, when the model page or the dashboard first needs it, and kept in memory. Training takes about two seconds. The evaluation report is written to a JSON file so that it can be shown on the model page and inspected.

E. Coding judge

The judge module exposes one function that receives the source code and the test cases and returns an overall status and a list of per-test results. The coding routes store each submission with its status and number of passed tests.

F. Admin import

The CSV importer reads the uploaded file as text, converts each row to a question, accepts an answer given either as option text or as a letter from A to D, validates the row with the same checks as the seed data, and inserts the valid rows. It reports how many rows were added, skipped as duplicates and rejected, with the first error messages.

4.4 BACKEND IMPLEMENTATION

The backend is one Flask application. Table 4.1 lists the main routes. Every route except login, registration and the home redirect requires a logged-in user, and the admin routes also require the administrator flag. A before-request hook checks the CSRF token of every POST request.

Table 4.1. Main routes of the Flask application

Route	Method	Purpose
/register, /login, /logout	GET, POST	Account creation, authentication and logout
/dashboard	GET	Readiness, averages, recommendation, weak and strong topics
/category/<name>	GET	Topics of a category with difficulty selector and level badges
/start-test	POST	Select 30 questions and begin a timed test
/test, /submit-test	GET, POST	Show the test; score, store and update ratings
/attempt/<id>	GET	Result and review of one stored test
/performance	GET	Per-topic accuracy, ability rating and score history
/mistakes	GET	Notebook of wrongly answered questions
/mentor	GET	Insights and the seven-day plan
/model	GET	Readiness-model evaluation report
/coding/<lang>/<topic>[/<problem>]	GET, POST	Problem list, problem page and judging
/admin, /admin/import, /admin/template.csv	GET, POST	Statistics, CSV import and the CSV template

4.5 DATABASE IMPLEMENTATION

A SQLite database stores everything the platform produces. Fig.4.2 shows the relationships and Table 4.2 lists the entities. An attempt belongs to a user, and each attempt has one answer row for every question shown. Because answers refer to questions, the accuracy of any topic can be computed by joining the two tables, which is the basis of the analytics. The ability table has a composite key of user and topic, so there is exactly one rating for each pair.

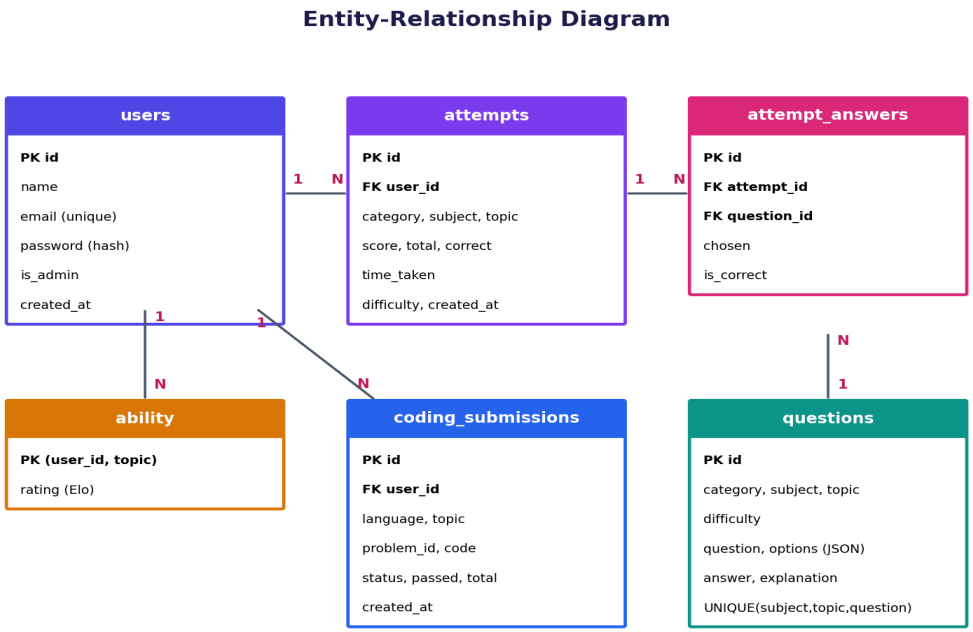


Fig.4.2. Entity-relationship diagram of the SQLite database

Table 4.2. Main database entities

Entity	Key attributes	Purpose
Users	id, name, email (unique), password hash, is_admin	Accounts and roles
Questions	id, category, subject, topic, difficulty, question, options (JSON), answer, explanation	The question bank; unique on subject, topic and question text
Attempts	id, user_id, category, subject, topic, score, total, correct, time_taken, difficulty, created_at	One row per completed test
attempt_answers	id, attempt_id, question_id, chosen, is_correct	One row per answered or skipped question
Ability	user_id, topic, rating	Elo rating of each learner for each topic
coding_submissions	id, user_id, language, topic, problem_id, code, status, passed, total	Every code submission with its verdict

4.6 USET INTERFACE

The interface is built from Jinja2 templates that extend one base layout. It uses a colour for each category (blue for Technical, pink for Verbal, amber for Aptitude and teal for Coding), gradient cards for the key numbers, colour-coded level and difficulty badges, and a layout that adapts to phone and desktop widths. The screens are shown in Chapter 5. Table 4.3 lists the main views and the data each one reads.

Table 4.3. Application views and their purpose

View	Purpose	Data source
Dashboard	Readiness label, average score, solved problems, recommended topic, weak and strong topics, recent tests	attempts, answers, ability, model
Topic picker	Choose topic and mode; shows level badge and number of questions	questions, ability
Test	30 questions, countdown, progress bar	questions
Result	Score ring, topic breakdown, review with explanations	attempts, answers
Performance	Table and charts of per-topic accuracy and score history	answers, ability
AI Mentor	Trend, advice and seven-day plan	answers, ability, model
Mistake notebook	Wrong answers with explanations	answers, questions
Model	Metrics, confusion matrix and feature importance	model report
Coding	Problem statement, editor, verdict table, recent submissions	problems, submissions
Admin	Statistics and CSV import	all tables

4.7 INTEGRATION

The modules were developed one at a time and then connected into the single application. Table 4.4 records the status of each module at the end of this phase.

Table 4.3. Module-wise implementation status

Module	Status	Remarks
1. Authentication and security	Completed	Hashing, CSRF, throttling, roles; tested
2. Question bank and generators	Completed	380 questions; every topic covered; validated
3. Test engine	Completed	30 distinct questions; timer; auto-submit
4. Adaptive engine (Elo)	Completed	Per-topic rating; direction of update tested
5. Analytics	Completed	Per-question accuracy; weak and strong topics
6. Readiness model	Completed	Trained on a simulated cohort; evaluated; to be retrained on real data
7. AI Mentor	Completed	Trend, advice and seven-day plan
8. Mistake notebook	Completed	Latest 40 distinct wrong questions
9. Coding judge	Completed	13 problems; Python judged automatically; other languages stored
10. Admin panel	Completed	Statistics and CSV import with validation
11. Database	Completed	Six tables with indexes and constraints
12. Web interface	Completed	Responsive; colour-coded; Chart.js charts

4.8 TESTING STRATEGY

The project has an automated suite of 24 tests, written with the Python unittest module and run with one command. The tests use a temporary database, so they do not touch real data. Table 4.5 shows what each group checks.

Table 4.5. Automated test suite

Test group	Tests	What is verified
Question bank	4	All questions valid; no duplicates; every topic has a question; a generated answer is arithmetically correct
Judge	5	Accepted, Wrong Answer, Runtime Error and Time Limit Exceeded verdicts; problem samples are consistent
Readiness model	3	Accuracy above 60% on hold-out data; needs at least three tests; a strong history gives High and a weak one Low

Test group	Tests	What is verified
Security	4	Missing CSRF token rejected; weak password rejected and hash stored; login required; wrong password rejected
Test engine	4	Full flow from login to review; at most 30 distinct questions; every topic gets exactly 30; a high rating gets Hard questions
Rating	1	A correct hard answer raises the rating and a wrong easy answer lowers it
Coding and admin	3	Python submission accepted; other language stored; CSV import adds a question; a non-admin receives HTTP 403

Together the tests exercise the registration, login, test, scoring, rating, review, analytics, coding and admin paths from the web layer down to the database.

4.9 SUMMARY

This chapter described the workflow of one test, the implementation of the modules, the Flask routes, the SQLite schema, the interface, the integration status and the automated tests. The next chapter evaluates the integrated system.

CHAPTER 5

5. EXPERIMENTAL ANALYSIS:

5.1 EXPERIMENTAL SETUP

The system was developed in Python with Flask, SQLite and scikit-learn, and tested with the automated suite and with measurements taken on the running application. The measurements were taken on a Linux machine with Python 3.12, using the Flask test client, so the response times do not include network delay. They show the cost of the application itself, not the experience of a student on a slow network.

No study with real students was carried out in this phase. The screenshots in Section 5.4 were produced with a demonstration account that took twelve tests with scripted answers, so the values on the screens describe the demonstration data and are not accuracy measurements of students.

Six kinds of measurement were used: the composition of the question bank, the results of the automated tests, the hold-out accuracy and cross-validation of the readiness model, the response time of the main operations, the judge time per submission, and a comparison of the new version with the first version.

5.2 PERFORMANCE EVALUATION

Hold-out accuracy is the share of students in the held-out part of the cohort for whom the model predicts the correct label. Cross-validation repeats the training five times with different splits and reports the mean and standard deviation. The macro F1 score is the average of the F1 scores of the three classes and does not favour the largest class. The confusion matrix shows which classes are confused. Response time is the average time to produce a page or complete an operation.

Table 5.1. Evaluation measures and what they show

Component	Measure	What it shows
Question bank	Count, difficulty mix, validation	Size and quality of the practice material
Test engine	Questions per test; duplicates	That each test has 30 distinct questions
Readiness model	Hold-out accuracy, 5-fold CV, macro F1, confusion matrix	How well the model separates Low, Medium and High on the simulated cohort
Coding judge	Time per submission; verdict correctness	Speed and behaviour of the judge

Component	Measure	What it shows
Application	Response time in milliseconds	Cost of the main pages and operations
Test suite	Tests passed	Correctness of the main behaviours

5.3 QUANTITATIVE RESULTS

Table 5.2 gives the module-level results.

Table 5.2. Performance evaluation of the proposed system

Module	Metric	Result
Question bank	Total questions	380 (Technical 153, Verbal 33, Aptitude 194)
Question bank	Generated aptitude questions	179 in 15 topics, all with worked explanations
Test engine	Questions per test	30 for every one of the 135 topics (verified by test)
Automated tests	Tests passed	24 of 24
Readiness model	Hold-out accuracy	69.5%
Readiness model	5-fold cross-validation	71.5% ± 1.3
Readiness model	Macro F1	0.693
Coding judge	Average time for 4 test cases	67 ms
Application	Training time of the model	2.28 s (once per start-up)

The question bank holds 380 questions, of which 183 are Easy, 119 Medium and 78 Hard. The validation checks found no invalid question and no duplicate, and every topic of the catalogue has at least one question. Because many technical topics have only a few questions of their own, the test engine fills the test from related topics, and the automated test confirmed that every one of the 135 topics yields exactly 30 distinct questions.

The readiness model reached a hold-out accuracy of 69.5% and a macro F1 of 0.693, with a cross-validation accuracy of 71.5% (standard deviation 1.3 points). The confusion matrix in Fig.5.1 shows that the model rarely confuses Low with High; almost all errors are between neighbouring classes, which is expected because the labels are cut from a continuous skill value. The most important features are accuracy (0.295), average score (0.237) and accuracy on hard tests (0.194). These results describe the simulated cohort and must not be read as an accuracy on real students.

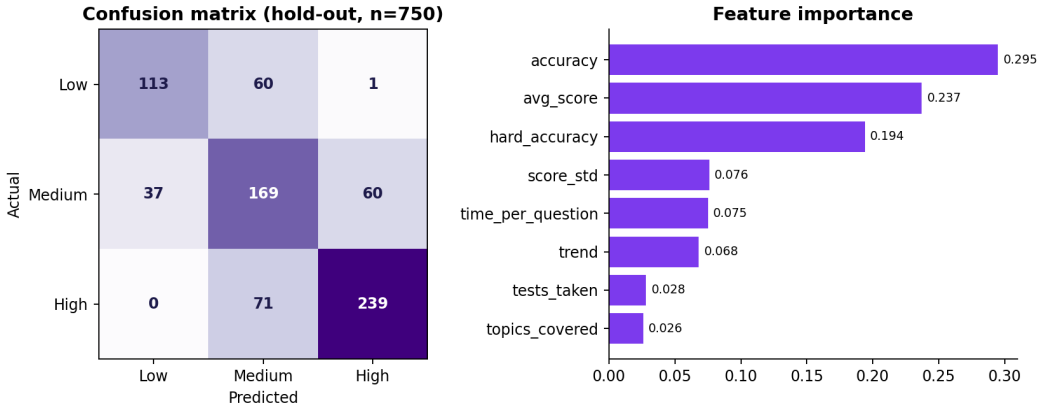


Fig.5.1. Confusion matrix and feature importance of the readiness model

Table 5.3 lists the average response times measured with the Flask test client over 30 repeated requests for each page and 200 repeated calls for the selector.

Table 5.3. Response time of the main operations

Operation	Average time	Note
Select 30 questions (adaptive)	0.3 ms (max 0.6 ms)	200 runs over random topics
Update 30 Elo ratings	0.03 ms	In-memory update of 30 answers
Dashboard page	1.8 ms	Includes readiness prediction
Topic picker (Technical)	2.6 ms	90 topic cards
Performance page	1.1 ms	Per-topic table
AI Mentor page	0.9 ms	Insights and plan
Seed 380 questions	25 ms	First start-up only
Judge: one submission, 4 tests	67 ms	Includes starting four Python processes

All measured operations complete in a few milliseconds, except the judge, which must start a Python process for every test case. These figures are adequate for a classroom of students on one laptop, but they were measured with a single user and say little about performance under many simultaneous users.

5.4 VISUAL RESULTS

The figures in this section show the screens of the platform. They were captured from the running application with a demonstration account. The values printed on the screens are demonstration data and are not accuracy measurements.

Fig.5.2 shows the login page. New students register with a name, email and a password of at least eight characters that contains letters and a digit.

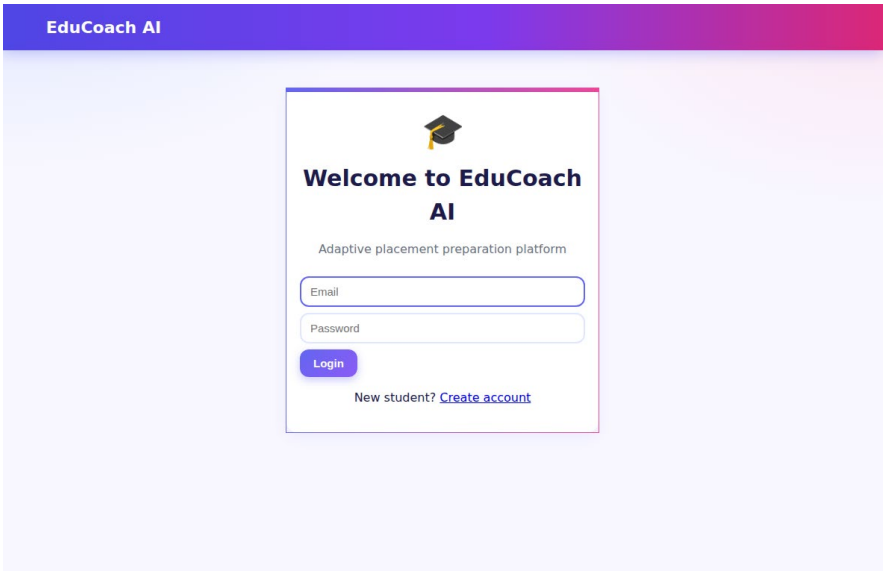


Fig.5.2. Login page

Fig.5.3 shows the dashboard. The four coloured cards show the readiness label with the model confidence, the average score, the number of coding problems solved, and the recommended next topic with a button that starts a 30-question test on it. Below them are the practice modules, the weak and strong topics and the recent tests.

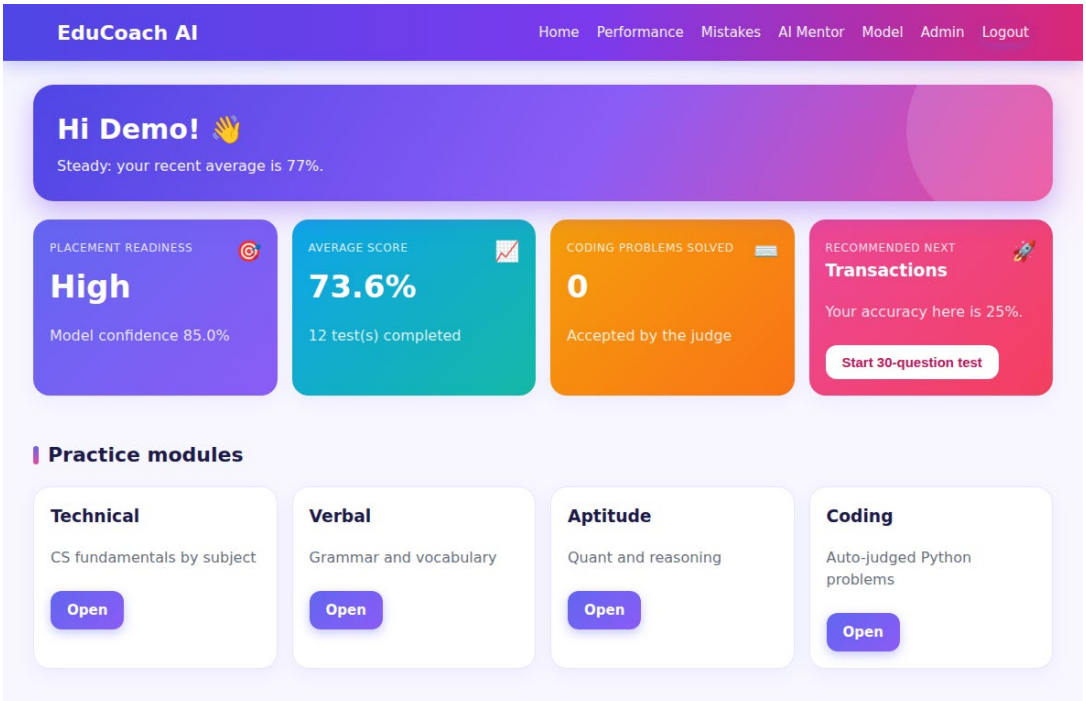


Fig.5.3. Student dashboard with readiness, average score and recommendation

Fig.5.4 shows the topic picker. Each topic card shows how many questions the topic has and, after the student has practised it, the ability level as a coloured badge. The student selects Adaptive, Easy, Medium or Hard mode and starts the test.

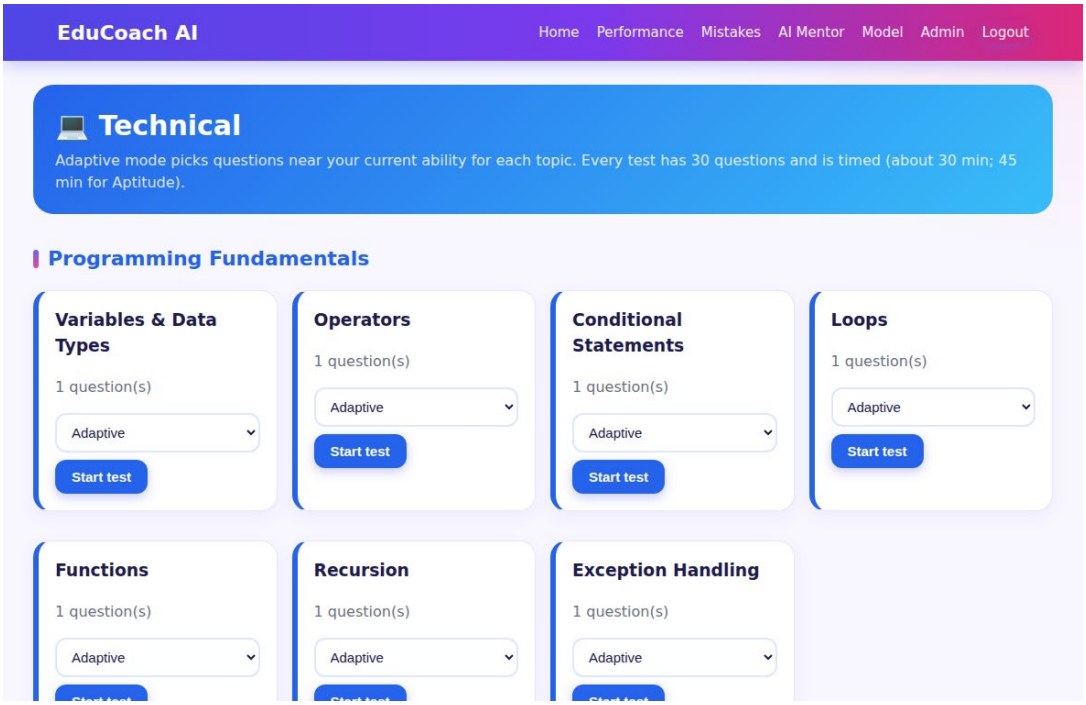


Fig.5.4. Topic picker with difficulty selector and level badges

Fig.5.5 shows a running test. The bar at the top shows the topic, the mode, the number of questions answered and the remaining time, and its colour fill grows as questions are answered. The timer turns red during the last minute.

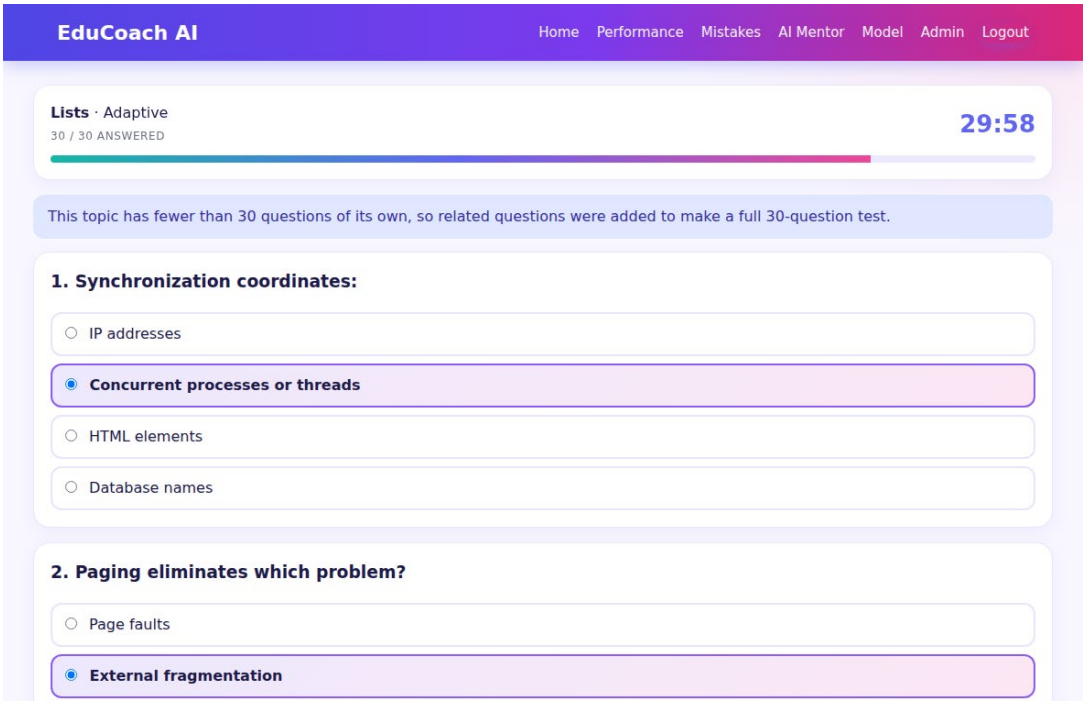


Fig.5.5. Timed 30-question test with countdown and progress bar

Fig.5.6 shows the result page. The ring shows the score and changes colour from red to amber to green. The chips give the correct answers per topic, and the review below lists every question with the student's answer, the correct answer and the explanation. Scores of 80% or more also trigger a short celebration animation.

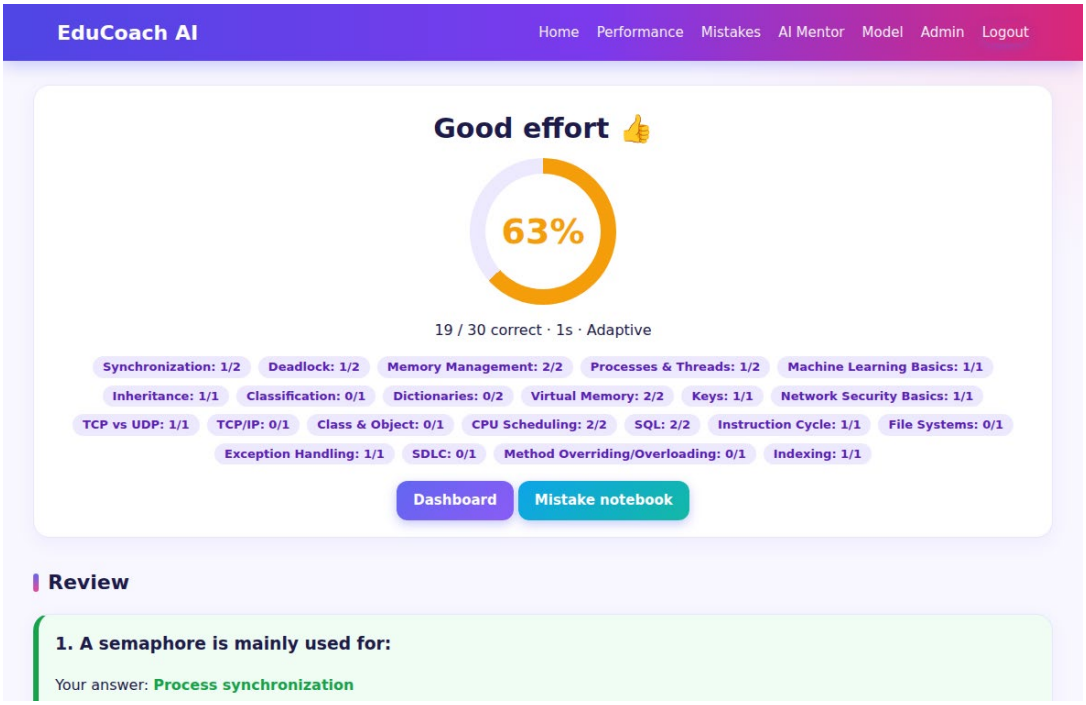


Fig.5.6. Result page with score ring, topic breakdown and answer review

Fig.5.7 shows the performance analytics. The table lists, for each topic that the student has answered, the accuracy, the number of answers and the ability level with its rating. The accuracy is coloured red below 60% and green from 80%. The page also draws two charts, a bar chart of accuracy by topic and a line chart of test scores over time, which need the Chart.js library from the internet and are therefore not visible in this offline capture.

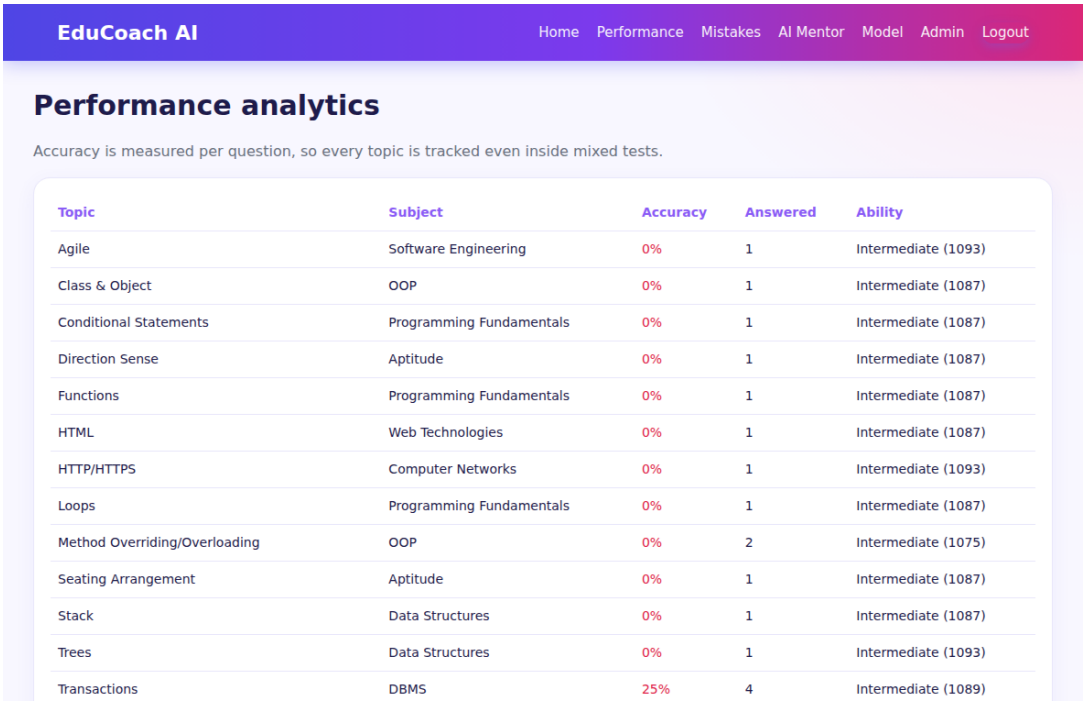


Fig.5.7. Performance analytics: per-topic accuracy and ability rating

Fig.5.8 shows the AI Mentor. The insights are tinted red for weak topics and green for strengths, and the seven-day plan below turns them into daily tasks.

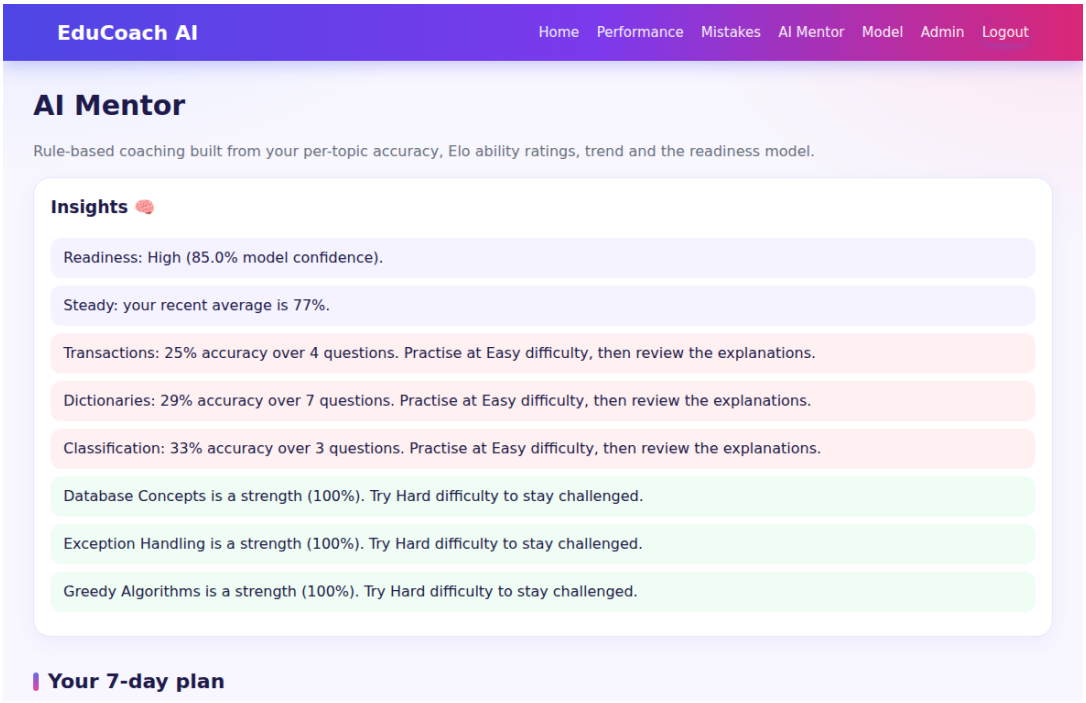


Fig.5.8. AI Mentor insights and 7-day study plan

Fig.5.9 shows the mistake notebook, which collects the questions answered wrongly so that they can be revised with their explanations.

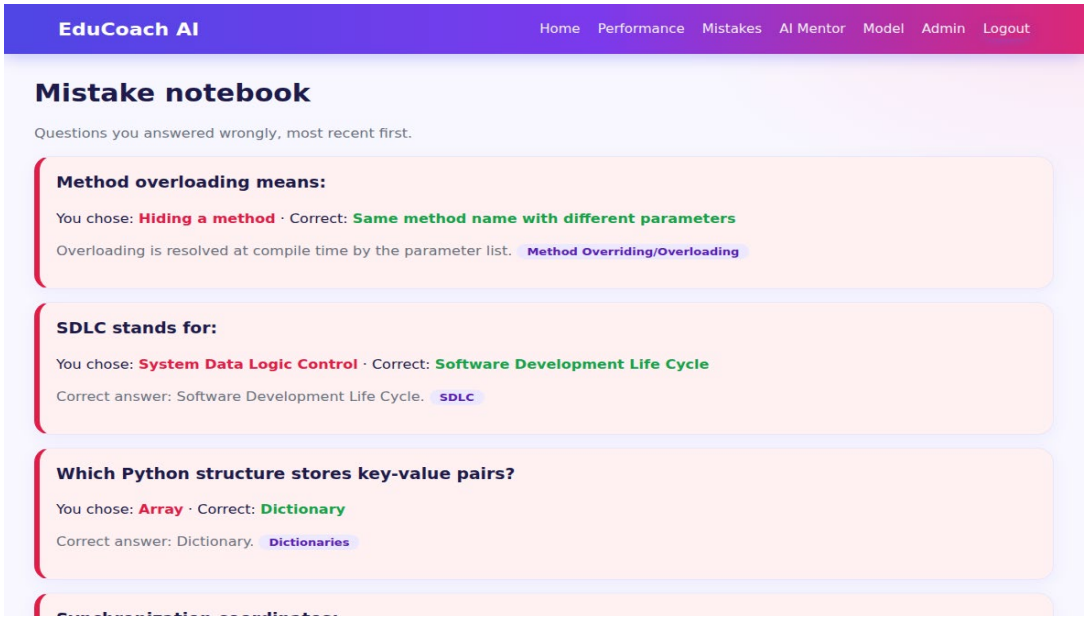


Fig.5.9. Mistake notebook

Fig.5.10 shows the readiness-model page, which states that the training data is simulated and shows the evaluation metrics, the confusion matrix and the feature importances.

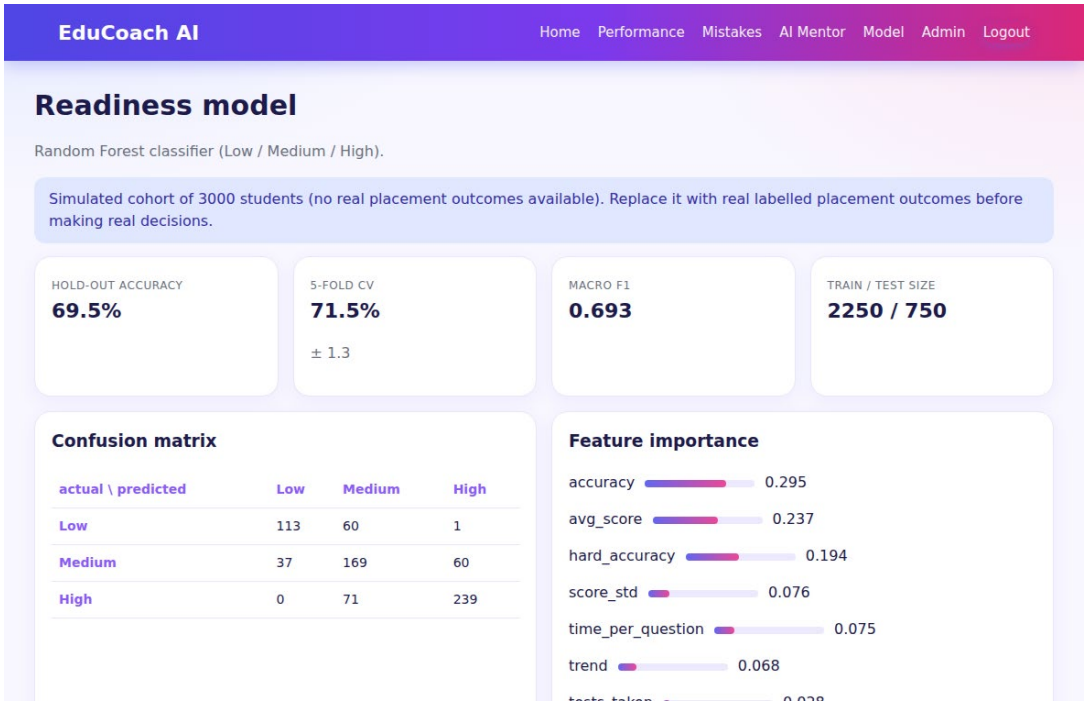


Fig.5.10. Readiness-model page with evaluation metrics

Fig.5.11 shows the coding judge. The student writes a Python solution, and the verdict table shows each test case as passed or failed. The first two test cases are visible; the others are hidden and are shown only as hidden tests.

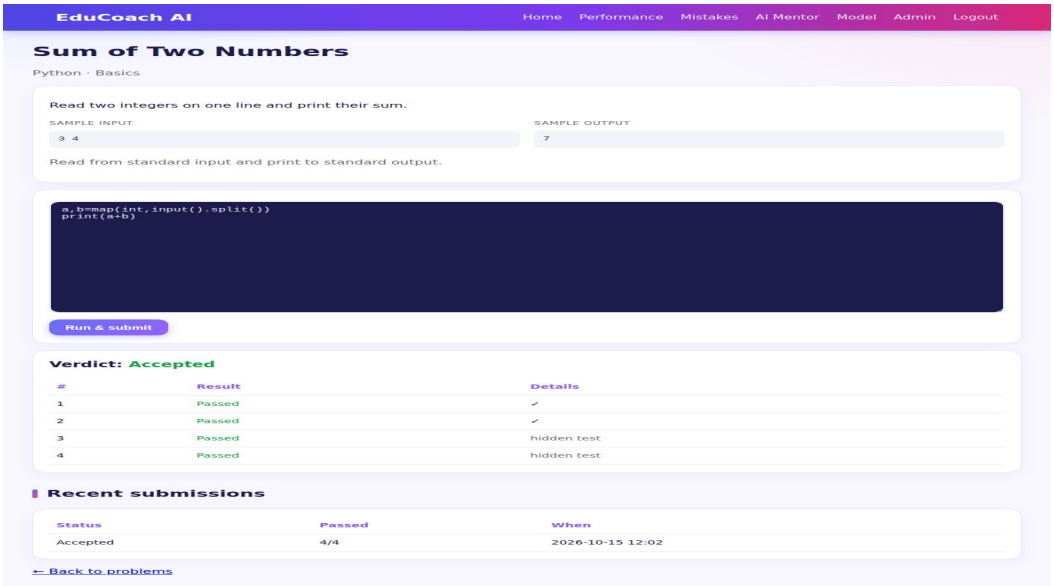


Fig.5.11. Coding judge showing the verdict for a submission

Fig.5.12 shows the admin panel, which gives the platform statistics and lets the administrator import questions from a CSV file.

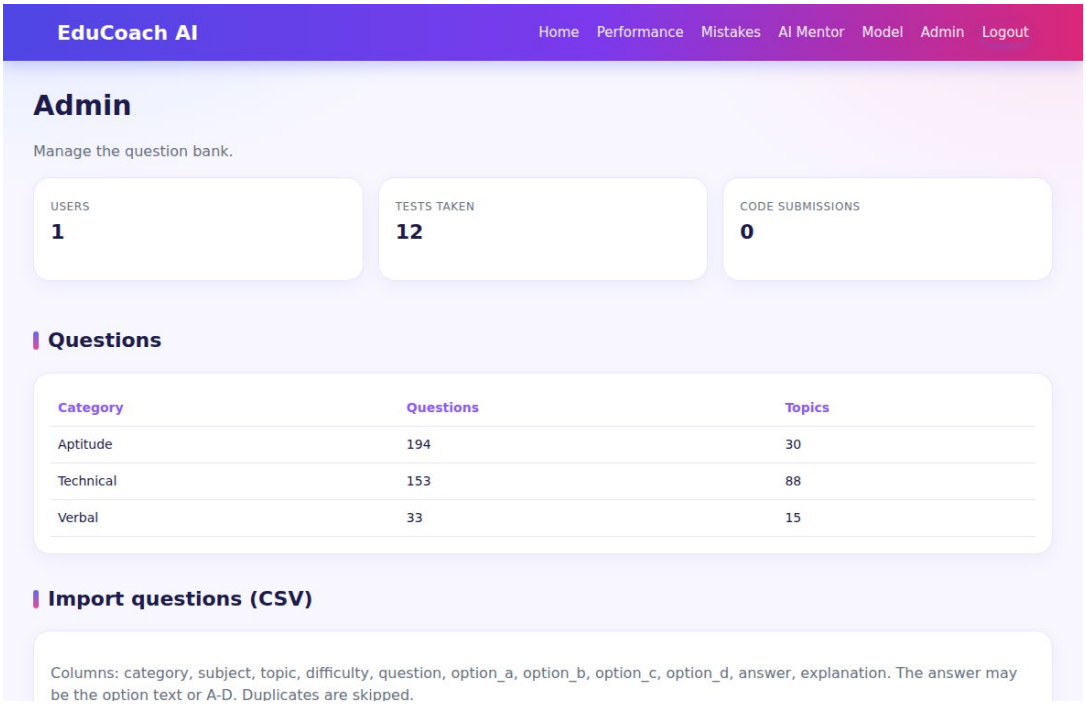


Fig.5.12. Admin panel with question statistics and CSV import

5.5 COMPARISON WITH EARLIER VERSION AND OTHER TOOLS

The first version of this project had the same screens but several weaknesses, which were found by reviewing its source code. Table 5.4 compares the two versions. The comparison is based on the code of the two versions.

Table 5.4. Comparison with the earlier version of the project

Aspect	First version	Present version
Questions	One question per topic, repeated up to 90 times with a number appended	380 distinct questions; no duplicates; validated
Test length	30 slots, mostly copies of the same question	30 distinct questions in every test
Adaptivity	Difficulty from the last score only	Elo ability rating per topic, used for selection
Analytics	Per-test score	Per-question, per-topic accuracy
Feedback	Score only	Review with explanations; mistake notebook
Readiness model	Random data; label is a threshold on the score	Simulated cohort with noisy hidden skill; evaluated; explained
Coding	Code stored only	Judged against hidden test cases (Python)
Passwords	Unsalted SHA-256	Salted hashes; throttling; CSRF protection
Admin	None	Statistics and validated CSV import
Automated tests	None	24 tests

Table 5.5 compares the coverage of typical preparation tools with the proposed system. The comparison is about coverage of features, not a ranking by quality, because the tools differ in size, purpose and data and no measured comparison was made.

Table 5.5. Coverage of typical preparation tools and the proposed system

Tool type	What it covers	Not covered
Coding practice sites (typical)	Large sets of coding problems with online judging	Aptitude and verbal practice; per-topic ability for all skills; readiness estimate
Aptitude question portals (typical)	Question lists and timed tests	Coding judge; adaptive selection; mentor plan
Printed books and notes	Explanations and worked examples	Timing, analytics, adaptation
Learning-science studies [7], [8], [9]	Evidence for testing and feedback	No software
Rating and tracing research [1], [3], [4]	Methods to estimate learner knowledge	A complete practice platform
EduCoach AI	Technical, Verbal, Aptitude and Coding practice; Elo adaptation; analytics; mentor; readiness model	Large-scale real-user validation; real placement data; multi-language judging

5.6 RESULT AND DISCUSSION

Taken together, the results show that the platform delivers its main promises. Every topic can be practised in a full 30-question test of distinct questions, the answers are stored one by one, and the stored answers feed the ratings, the analytics, the mistake notebook, the mentor and the readiness prediction. All 24 automated tests pass, which gives confidence that the behaviours described in this report are those of the code.

The aptitude generator is the most important change for the long term, because it supplies new questions with correct answers and explanations without manual work. The curated technical and verbal questions are fewer, and many topics still rely on questions from related topics to fill a test. This is the main practical limit of the current version and it can be improved by the CSV import.

Reading the model result

A hold-out accuracy of 69.5% on three classes is clearly better than the 33% of random guessing, but it is not high, and this is expected. The labels in the simulated data contain noise, so even a perfect model could not reach 100%. The result shows that the pipeline works and that the model uses sensible features, not that the model can predict the placement outcome of a real student.

Error analysis

Question bank. The bank is large in aptitude and smaller in verbal and technical topics. A learner who practises one technical topic repeatedly will see its few questions often, although the selector avoids recently seen questions when it can.

Readiness model. Most errors are between neighbouring classes, Low and Medium or Medium and High. The data is simulated, so errors on real students could be larger or of a different kind.

Judge. The judge detected all four verdict types in the tests. Because it runs the submitted code on the host machine, it must not be exposed to untrusted users without further isolation.

Threats to validity

The first threat is that the readiness model was trained and evaluated on simulated data, so its accuracy says nothing yet about real students. The second is that the response times were measured with one user and the test client, not under load. The third is that no study with students has shown that the adaptive selection and the feedback improve their results, although the educational literature [7], [8] supports the approach. The fourth is that the question bank has not been reviewed by a large number of subject experts.

Table 5.6 collects the results and the limiting factors in one place.

Table 5.6. Results and the limiting factors reported for each module

Module	Result	Limiting factor
Question bank	380 questions, all valid	Few questions per technical and verbal topic
Test engine	30 distinct questions for every topic	Mixed tests for topics with few questions
Adaptive engine	Update direction verified by test	Not yet validated against student outcomes
Readiness model	69.5% hold-out accuracy	Simulated training data
Coding judge	Four verdicts detected; 67 ms per submission	Not a full sandbox; Python only
Application	Milliseconds per page	Measured for a single user only

5.7 SUMMARY

This chapter described the experimental set-up and the measures used, presented the results for the question bank, the test engine, the readiness model and the response times, showed the screens of the platform, and compared the present version with the first version and with typical tools. The final chapter gives the conclusion, the limitations and future work.

CHAPTER 6

6. CONCLUSION AND FUTURE WORK:

6.1 CONCLUSION

This project built EduCoach AI, an adaptive placement-preparation platform that brings Technical, Verbal, Aptitude and coding practice into one web application with one learner profile. It combines a validated question bank with generated aptitude questions, timed tests of exactly 30 distinct questions, an Elo-style ability rating for every topic, per-question analytics, answer review with explanations, a mistake notebook, an AI Mentor with a seven-day plan, a Random Forest readiness model and an auto-judged coding module, protected by salted password hashing, CSRF protection and login throttling.

The measured results support the design. The bank holds 380 questions with no invalid question or duplicate, every one of the topics gives a full 30-question test, all 24 automated tests pass, the readiness model reaches 69.5% hold-out accuracy on its simulated cohort, and the main pages respond in a few milliseconds.

Contributions

The work makes three contributions. The first is the integration itself: one learner profile connects practice in four skill areas to adaptive selection, analytics, feedback and a readiness estimate. The second is the content and quality machinery: generators that produce aptitude questions with correct answers and explanations, validation rules shared by seeding, CSV import and tests, and a suite of 24 automated tests. The third is an honest treatment of the machine-learning part: the model is trained on a simulated cohort, evaluated with hold-out data, cross-validation, a confusion matrix and feature importances, and clearly labelled in the application so that it can be replaced by a model trained on real data.

6.2 LIMITATIONS AND CHALLENGES

The readiness model is trained on simulated data, because no dataset of real students with known placement outcomes was available. Its reported accuracy therefore describes the simulation and not real students. Many technical and verbal topics have only a few questions of their own, so tests on those topics are filled with related questions. The code judge runs submissions on the host machine in a separate process with limits, but it is not a full sandbox, and it judges Python only.

The rating parameters (the starting rating of 1100, the question ratings 1000, 1200 and 1400, and K = 20) were chosen by reasoning and not fitted to student data. The platform uses SQLite and was measured with a single user, so it is intended for one classroom or one learner and not for hundreds of simultaneous users.

Each limitation points to a direction for improvement. The model limit is removed by collecting real outcomes. The question limit is removed by growing the bank through the CSV import and expert review. The judge limit is removed by running it in a container. Some limits are connected: the shortage of real data affects both the readiness model and the choice of the rating parameters, and both would be improved by a pilot study with students.

6.3 FUTURE ENHANCEMENT

The remaining work includes, but is not limited to, the following.

- Run a pilot with a class of students and use the recorded answers and the real placement results to train and validate the readiness model and to fit the rating parameters.
- Grow the question bank, especially for technical and verbal topics, with expert-reviewed questions, and calibrate the difficulty of each question from the answers of real students.
- Run the judge inside a container or a separate service, add Java, C, C++ and JavaScript judging, and add more problems with a stronger set of hidden tests.
- Add email verification, password reset and a teacher view that shows a whole class, so that a training and placement cell can follow its students.
- Replace SQLite with a server database and deploy behind a production web server, to support many simultaneous users.
- Add spaced repetition for the mistake notebook, and explain the readiness prediction to the student feature by feature.

Possible uses

The platform is aimed at college training and placement cells and at individual students. In such a setting it replaces the routine of moving between several sites and books with a single place that remembers what the student knows and suggests what to practise next. The same architecture could be adapted to other entrance or recruitment examinations by changing the catalogue of topics and the question bank, which are data and not code.

6.4 SUMMARY

This chapter concluded that one adaptive platform can bring four preparation skills, feedback and readiness estimation together on an ordinary laptop. It listed the limitations, mainly the simulated training data, the small number of questions in some topics and the limits of the judge, and set out directions for further work.

CHAPTER 7

REFERENCES:

[1] Pelánek, R. (2016). Applications of the Elo rating system in adaptive educational systems. Computers & Education, 98, 169–179.

[2] Elo, A. E. (1978). The Rating of Chessplayers, Past and Present. New York: Arco Publishing.

[3] Corbett, A. T., & Anderson, J. R. (1994). Knowledge tracing: Modeling the acquisition of procedural knowledge. User Modeling and User-Adapted Interaction, 4(4), 253–278.

[4] Piech, C., et al. (2015). Deep knowledge tracing. In Advances in Neural Information Processing Systems (NeurIPS), 28.

[5] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.

[6] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[7] Roediger, H. L., III, & Karpicke, J. D. (2006). Test-enhanced learning: Taking memory tests improves long-term retention. Psychological Science, 17(3), 249–255.

[8] Hattie, J., & Timperley, H. (2007). The power of feedback. Review of Educational Research, 77(1), 81–112.

[9] Black, P., & Wiliam, D. (1998). Assessment and classroom learning. Assessment in Education: Principles, Policy & Practice, 5(1), 7–74.

[10] Lord, F. M. (1980). Applications of Item Response Theory to Practical Testing Problems. Hillsdale, NJ: Lawrence Erlbaum Associates.

[11] OWASP Foundation. (n.d.). OWASP Top Ten Web Application Security Risks. Retrieved from https://owasp.org/www-project-top-ten/

[12] Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2nd ed.). Sebastopol, CA: O'Reilly Media.

CHAPTER 8

8. SOURCE CODE:

```
import json, os
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, f1_score
from sklearn.model_selection import cross_val_score, train_test_split

FEATURES = ["accuracy", "avg_score", "time_per_question", "score_std", "trend",
"tests_taken", "hard_accuracy", "topics_covered"]
LABELS = ["Low", "Medium", "High"]
MIN_TESTS = 3

def simulate_cohort(n=3000, seed=42):
    r = np.random.default_rng(seed)
    skill = r.normal(0, 1, n)
    tests = r.integers(3, 30, n)
    accuracy = np.clip(0.62 + 0.17 * skill + r.normal(0, 0.07, n), 0.15, 0.99)
    hard_acc = np.clip(accuracy - 0.12 + 0.05 * skill + r.normal(0, 0.08, n), 0.05, 0.99)
    avg_score = np.clip(accuracy * 100 + r.normal(0, 4, n), 5, 100)
    tpq = np.clip(42 - 6 * skill + r.normal(0, 9, n), 8, 120)
    std = np.clip(14 - 3 * skill + r.normal(0, 4, n), 1, 40)
    trend = np.clip(0.04 * skill + r.normal(0, 0.08, n), -0.4, 0.5)
    topics = np.clip((tests * r.uniform(0.3, 1.0, n)).astype(int) + 1, 1, 60)
    readiness = skill + 0.8 * trend + 0.15 * np.log(tests) + r.normal(0, 0.45, n)
    y = np.where(readiness < -0.45, "Low", np.where(readiness < 0.65, "Medium", "High"))
    X = np.column_stack([accuracy, avg_score, tpq, std, trend, tests, hard_acc, topics])
    return X, y

_cache = {}

def get_model():
    if "m" in _cache: return _cache["m"], _cache["report"]
    X, y = simulate_cohort()
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=7, stratify=y)
    m = RandomForestClassifier(n_estimators=200, max_depth=8, min_samples_leaf=5,
random_state=42)
    m.fit(Xtr, ytr); pred = m.predict(Xte)
    cv = cross_val_score(RandomForestClassifier(n_estimators=100, max_depth=8,
min_samples_leaf=5, random_state=1), X, y, cv=5)
    report = dict(
        data="Simulated cohort of 3000 students (no real placement outcomes available)",
        holdout_accuracy=round(float(accuracy_score(yte, pred)), 3),
        macro_f1=round(float(f1_score(yte, pred, average="macro")), 3),
        cv_mean=round(float(cv.mean()), 3), cv_std=round(float(cv.std()), 3),
        labels=LABELS, confusion=confusion_matrix(yte, pred, labels=LABELS).tolist(),
```

```
        importances=sorted(zip(FEATURES, [round(float(v), 3) for v in m.feature_importances_]),
key=lambda t: -t[1]),
        train_size=int(len(ytr)), test_size=int(len(yte)),
    )
    _cache["m"], _cache["report"] = m, report
    try:
        with open(os.path.join(os.path.dirname(__file__), "data", "model_report.json"), "w") as f:
json.dump(report, f, indent=2)
    except OSError:
        pass
    return m, report

def features_from_attempts(rows, topics_covered):
    """rows: attempts (newest first) with score,total,correct,time_taken,difficulty. Returns feature
list or None."""
    if len(rows) < MIN_TESTS: return None
    rows = list(reversed(rows))  # oldest first
    scores = np.array([r["score"] for r in rows], dtype=float)
    acc = float(np.mean([r["correct"] / r["total"] for r in rows if r["total"]]))
    tpq = float(np.mean([r["time_taken"] / max(r["total"], 1) for r in rows]))
    k = min(3, len(scores) // 2) or 1
    trend = float((scores[-k:].mean() - scores[:k].mean()) / 100)
    hard = [r["correct"] / r["total"] for r in rows if r["difficulty"] == "Hard" and r["total"]]
    hard_acc = float(np.mean(hard)) if hard else max(acc - 0.12, 0.05)
    return [acc, float(scores.mean()), tpq, float(scores.std()), trend, len(rows), hard_acc,
topics_covered]

def predict(features):
    m, _ = get_model()
    proba = m.predict_proba([features])[0]
    idx = int(np.argmax(proba))
    return m.classes_[idx], round(float(proba[idx]) * 100, 1), {c: round(float(p) * 100, 1) for c, p
in zip(m.classes_, proba)}

###============================================================###
import os, subprocess, sys, tempfile

TIME_LIMIT = float(os.environ.get("JUDGE_TIME_LIMIT", "3"))
MEMORY_MB = int(os.environ.get("JUDGE_MEMORY_MB", "256"))
MAX_CODE = 20000

def _limits():  # POSIX only
    try:
        import resource
        resource.setrlimit(resource.RLIMIT_AS, (MEMORY_MB * 1024 * 1024,) * 2)
        resource.setrlimit(resource.RLIMIT_CPU, (int(TIME_LIMIT) + 1,) * 2)
    except Exception:
        pass
```

```
def run_python(code, tests):
    """Return (overall_status, results). Each result: dict(n, hidden, verdict, passed, input, expected,
got)."""
    if len(code) > MAX_CODE: return "Rejected", [dict(n=1, hidden=False, verdict="Code too
long", passed=False, input="", expected="", got="")]
    results = []
    with tempfile.TemporaryDirectory() as d:
        path = os.path.join(d, "solution.py")
        with open(path, "w", encoding="utf-8") as f: f.write(code)
        for i, (inp, exp) in enumerate(tests, 1):
            hidden = i > 2  # first two tests are visible, the rest hidden
            kw = dict(input=inp + "\n", capture_output=True, text=True, timeout=TIME_LIMIT,
cwd=d)
            if os.name == "posix": kw["preexec_fn"] = _limits
            try:
                p = subprocess.run([sys.executable, "-I", path], **kw)
                out = p.stdout.strip()
                if p.returncode != 0: verdict, ok, got = "Runtime Error", False,
(p.stderr.strip().splitlines() or [""])[-1][:200]
                elif out == exp.strip(): verdict, ok, got = "Passed", True, out
                else: verdict, ok, got = "Wrong Answer", False, out[:200]
            except subprocess.TimeoutExpired:
                verdict, ok, got = "Time Limit Exceeded", False, ""
            results.append(dict(n=i, hidden=hidden, verdict=verdict, passed=ok, input=inp,
expected=exp, got=got))
    if all(r["passed"] for r in results): status = "Accepted"
    else: status = next(r["verdict"] for r in results if not r["passed"])
    return status, results
###============================================================================###

import os

import sqlite3

from flask import Flask, g

app = Flask(__name__)

BASE = os.path.dirname(os.path.abspath(__file__))

app.config["DB"] = os.path.join(BASE, "data", "educoach.db")

def get_db():

if "db" not in g:

g.db = sqlite3.connect(app.config["DB"])

g.db.row_factory = sqlite3.Row

g.db.execute("PRAGMA foreign_keys=ON")
```

```
return g.db
def close_db(error=None):
db = g.pop("db", None)
if db is not None:
db.close()
app.teardown_appcontext(close_db)
SCHEMA = """
CREATE TABLE IF NOT EXISTS users(
id INTEGER PRIMARY KEY, name TEXT NOT NULL,
email TEXT UNIQUE NOT NULL, password TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS attempts(
id INTEGER PRIMARY KEY, user_id INTEGER,
category TEXT, topic TEXT, score REAL,
total INTEGER, correct INTEGER, difficulty TEXT
);
CREATE TABLE IF NOT EXISTS ability(
user_id INTEGER, topic TEXT, rating REAL,
PRIMARY KEY(user_id, topic)
);
"""
```

OUTPUT:

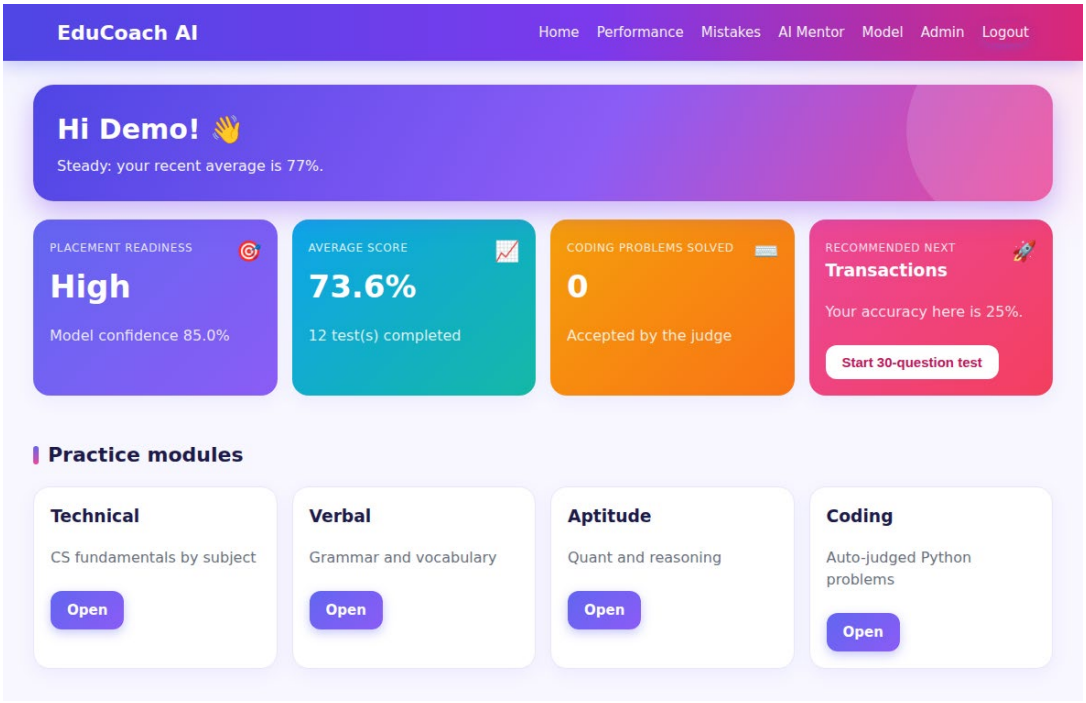


Fig.8.1. Student dashboard with readiness, average score and recommendation

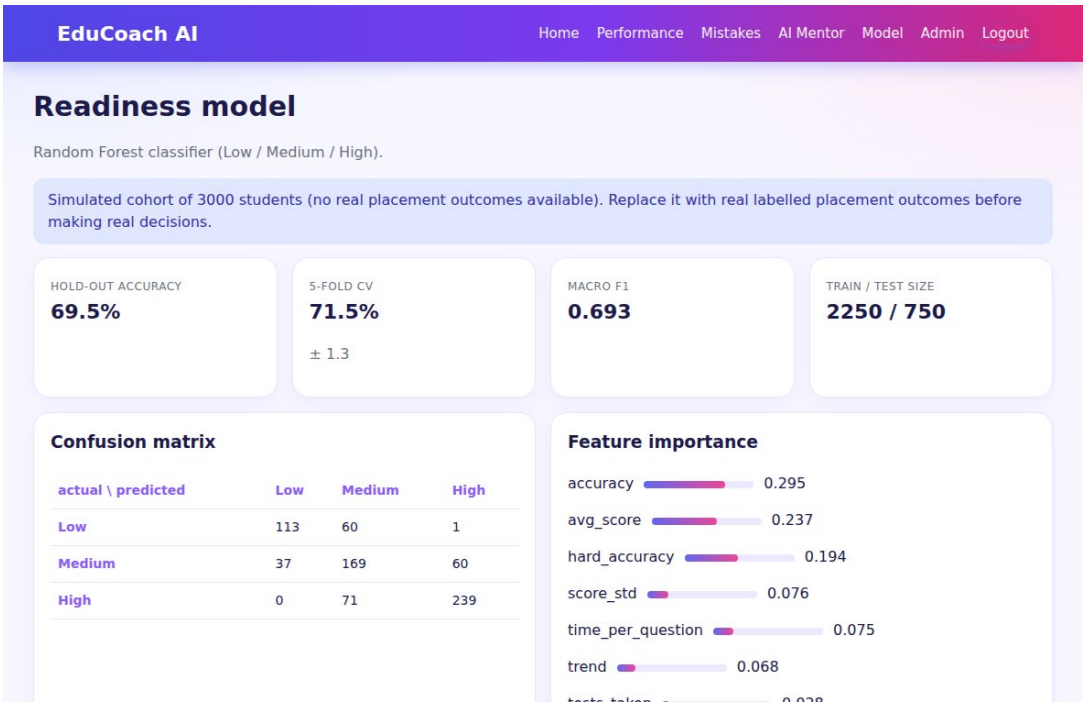


Fig.8.2. Readiness-model page with evaluation metrics

CONFERENCE APPLICATION DETAILS

The work reported here was written up as a conference paper and submitted through Microsoft CMT. The details below are taken from the submission-creation email received from Microsoft CMT, reproduced as evidence on the final page of this report.

Table 9.1. Conference submission details

Field	Details
Submission system	Microsoft CMT
Track name	Track 01: Computational Intelligence & Advanced Data System
Paper ID	1343
Paper title	Adaptive AI -Powered Placement Preparation Platform Using Multi-Agent Intelligence for Personalized Career Readliness
Submission status	Submission created (email: “The following submission has been created.”)
Created on	Mon,19 Oct 2026 12:07:43 GMT
Last Modified	Mon,19 Oct 2026 12:07:43 GMT
Authors (as listed in the email)	sathyasri6075@gmail.com
Authors (as listed in the paper)	Sathya Sri (student); Dr.. Sangeetha (Assistant Professors), Department of Computer Science and Engineering(Artificial Intelligence and Machine Learning), VISTAS, Chennai, India

CONFERENCE APPLICATION / SUBMISSION EVIDENCE

